Smart Energy Grid Analytics

Ingestion, storage benchmarking, continuous aggregates, dashboards and next-day demand forecasting

Advanced Big Data Analytics – Assignment I

Group members
HITAYEZU James ID: 101381
Joseph Courbon ID: 101493
Mylene MANIRAKIZA ID: 101403

Code repository
https://github.com/Jocos12/Smart_Energy_Grid_Analytics.git

Issued 1 October 2026 · Due 15 October 2026, 11:59 PM CAT
All dates and times are in CAT (Africa/Kigali, UTC+02:00)

Table of contents

Executive summary 4

1 Build the ingestion pipeline 5

1.1 Confirm the environment 5

1.2 Create the database structures 8

1.3 Capture and publish a pilot stream 9

2 Capture and prepare the historical dataset 10

2.1 Capture and load the complete stream 10

2.2 Investigate data quality 10

2.3 Clean and preserve a consistent benchmark 10

3 Compare chunk intervals 11

3.1 Create and verify three configurations 11

3.2 Implement and time the five workloads 12

3.3 Explain the chunk results 14

4 Compare compression strategies 15

4.1 Create the two compressed configurations 15

4.2 Measure storage and compression cost 16

4.3 Repeat the query comparison 17

4.4 Select a storage configuration 19

5 Investigate the utility data 20

5.1 Identify demand patterns 20

5.2 Compare regions and customer types 21

5.3 Assess reporting quality 24

5.4 Investigate abnormal consumption 25

5.5 Management recommendation (≤ 500 words) 27

6 Create and investigate continuous aggregates 29

6.1 Create the three aggregate views 29

6.2 Test freshness and delayed arrivals 32

7 Build and compare dashboards 34

7.1 Build the Raw dashboard 34

7.2 Build the Aggregate dashboard 35

7.3 Verify matching results 36

7.4 Measure performance and judge the trade-off 38

8 Forecast the next day 42

8.1 Prepare chronological data and features . . . 42
8.2 Evaluate the previous-day baseline . . . 43
8.3 Train and evaluate one regression model . . . 43
8.4 Judge forecast usefulness . . . 44

9 Submit the report and code . . . 47
9.1 Organise the report . . . 47
9.2 Make the work reproducible . . . 47
9.3 Document contributions . . . 47

Executive summary

This report documents the analytics chain for a 4,000-meter electricity network: capture and ingestion of the simulated smart-meter stream into PostgreSQL/TimescaleDB, data-quality assessment, demand analysis, anomaly detection, continuous aggregates, a Raw-versus-Aggregate dashboard comparison, and a day-ahead regional forecast with a previous-day baseline and a gradient-boosting regression model. Sections follow the order of the assignment brief; each figure is a screenshot of the executed command or dashboard, with the taskbar and clock visible.

Key results

Topic	Result	Section
Cleaned dataset	10,643,735 unique readings · 4,000 meters · 3,853,704.35 kWh (3,853.7 MWh)	§2.3, §6.1
Reporting quality	≈ 98.99 % completeness (missing 108,265 of 10,752,000 planned readings = 1.007 %)	§2.2, §5.3
Demand pattern	Weekday/weekend cycle: 141–152 MWh/day on weekdays vs. 112–119 MWh at weekends (−21 %); highest 15-minute demand 7,993.7 kW on 28 September at 13:00	§5.1
Customer mix	Industrial meters (10 % of meters) use 58 % of the energy, 17.5× more per meter than residential ones; the regional differences follow the industrial share	§5.2
Anomaly detection	40 / 40 injected events detected · 0 false flags (0.000000 %) · 474 flagged readings	§5.4
Continuous aggregates	3 views: 26,880 / 6,720 / 112,000 rows · ≈ 18 MB in total · totals identical to the raw data	§6.1
Chunk intervals	Fastest median: 3 h chunks on Q1, 1 day on Q2, 1 week on Q3 and Q4 (Q5: no clear difference); 3 h chunks are 5.9× slower on Q3 (225 chunks). 1 day chosen as the best compromise	§3.2–3.3
Compression	1,344 MB → 439 MB (ratio 3.06) without segmentation, 502 MB (ratio 2.68) segmented by meter_id; compression time 27.2 s vs 46.9 s; Q4 plan 115.7 ms vs 1.257 ms	§4.2–4.4
Dashboard speed-up	Median whole-dashboard time over 10 refreshes: 2,780.3 ms (Raw) → 610.8 ms (Aggregate), ≈ 4.6× faster; query time of the heavy panels 80–103× lower	§7.4
Dashboard consistency	All five panels identical between Raw and Aggregate: max energy difference 1.2×10^{-8} kWh, completeness difference 0	§7.3
Forecast	Previous-day baseline on 1,680 test predictions: MAE 42.708 kWh, RMSE 79.403 kWh. Gradient-boosting model (C3): MAE 7.489 kWh, RMSE 11.299 kWh (−82.5 % / −85.8 %)	§8.2–8.4

1 Build the ingestion pipeline

1.1 Confirm the environment

The assignment database `energy_grid` runs on a native Windows installation of PostgreSQL 18 (service `postgresql-x64-18`, state RUNNING), reached through `psql.exe` from `C:\Program Files\PostgreSQL\18\bin`. TimescaleDB 2.30.2 is enabled in this database and the session time zone is Africa/Kigali so that daily buckets align with CAT midnight. EMQX 5.8.9 runs in a Docker container. The table lists the recorded environment; the screenshots follow it.

Item	Recorded value
CPU	13th Gen Intel(R) Core(TM) i7-1355U – 10 cores, 12 logical processors
RAM	15.7 GB
Operating system	Microsoft Windows 11 Pro, version 10.0.26200
Storage	KIOXIA KBG50ZNV1T02, SSD, 1,024,209,543,168 bytes (≈ 1 TB)
Python	3.12.1
PostgreSQL	18.6 on x86_64-windows, compiled by msvc-19.44.35229, 64-bit (*SELECT version()*)
Deployment of PostgreSQL	Native Windows service (not a container or VM), no CPU or RAM limit other than the machine's 12 logical processors and 15.7 GB; connection to localhost, port 5432 (::1)
Database / user	energy_grid / postgres (superuser = on), password authentication, SSL off, protocol 3.0
TimescaleDB	2.30.2 (Community Edition), schema public (*\dx timescaledb* and *pg_extension*)
EMQX	5.8.9, Docker image emqx/emqx:5.8.9, container *emqx* with ports 1883, 8083, 8084, 8883 and 18083 published; no --cpus or --memory option was passed; node emqx@172.17.0.2 started; TCP test of port 1883 succeeded
Server settings	shared_buffers = 4023MB · work_mem = 10730kB · max_parallel_workers_per_gather = 6 · jit = off
Time zone	Session time zone Africa/Kigali (CAT, UTC+02:00); the Windows time zone is UTC+02:00 (Israel Standard Time, DST off), the same offset as CAT. All timestamps in this report are CAT.
Clock check	Get-Date returned 2026-10-10 19:07:09, equal to the taskbar clock (7:07 PM). The Windows Time service was not started when checked (w32tm /resync and /query /status returned error 0x80070426), so network synchronisation could not be demonstrated.

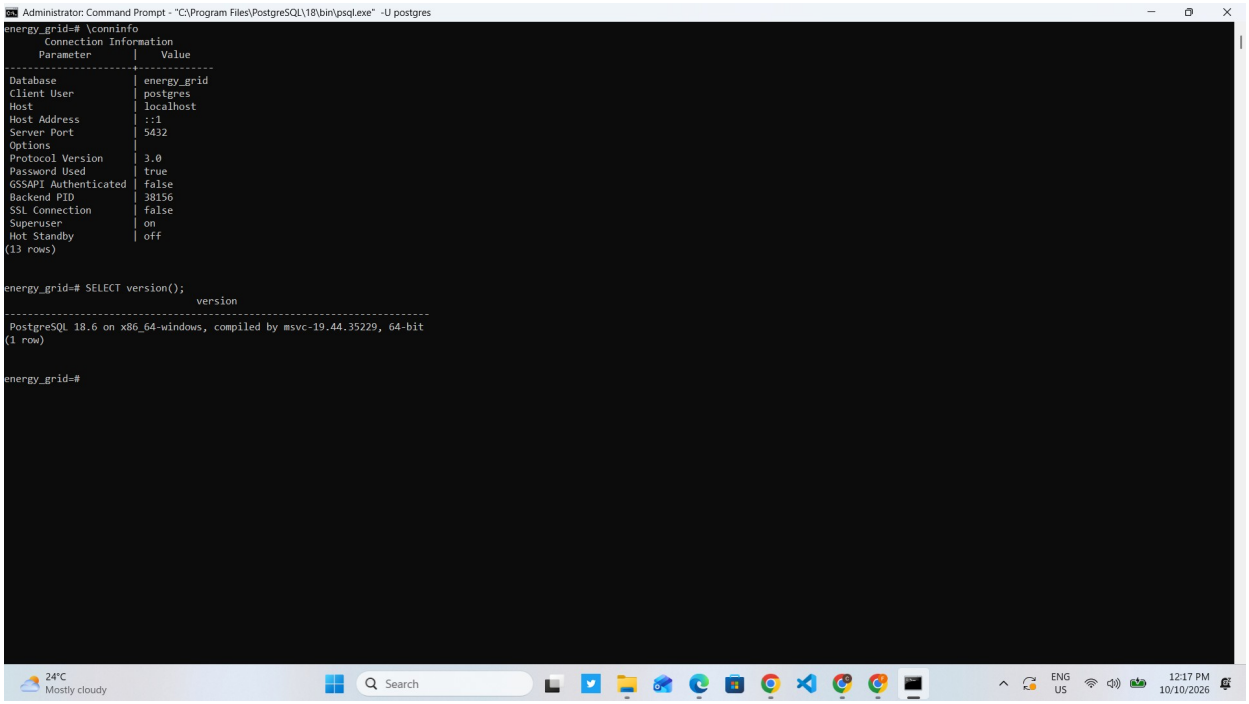


Figure 1 – Connection to the assignment database (\conninfo) and PostgreSQL version (10 Oct 2026, 12:17 PM).

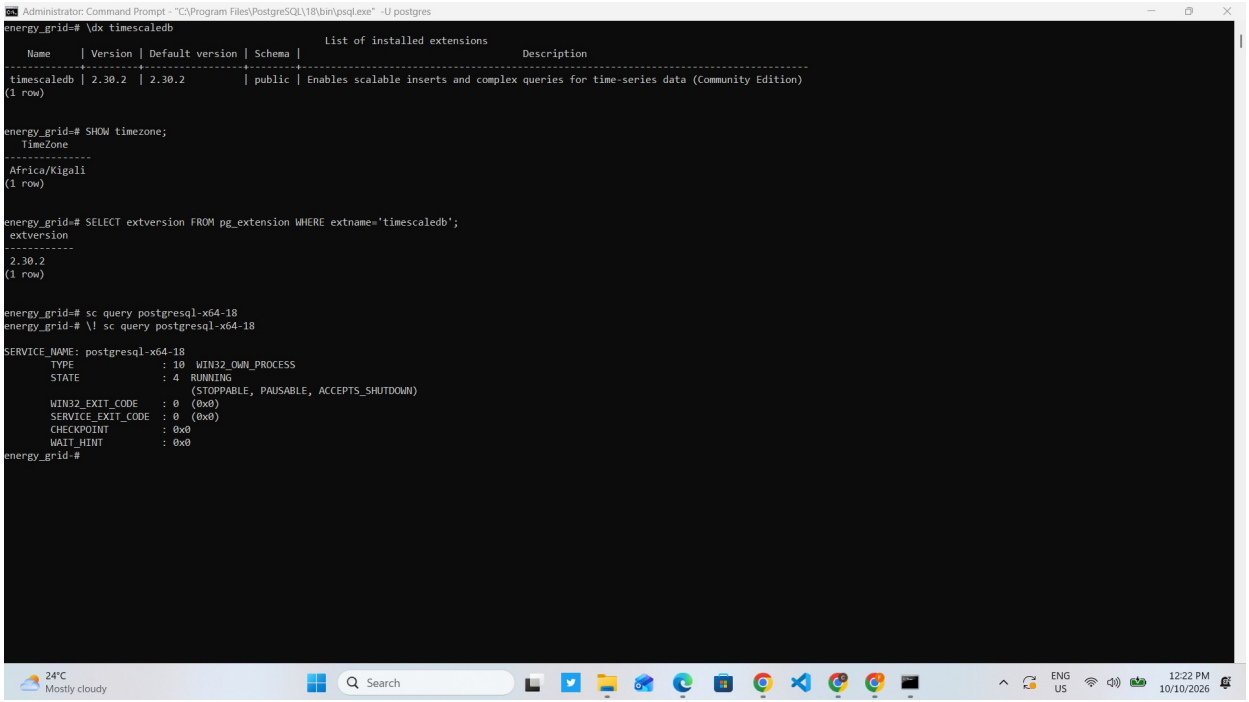


Figure 2 – TimescaleDB extension (version 2.30.2), session time zone Africa/Kigali and the PostgreSQL service state (10 Oct 2026, 12:22 PM).

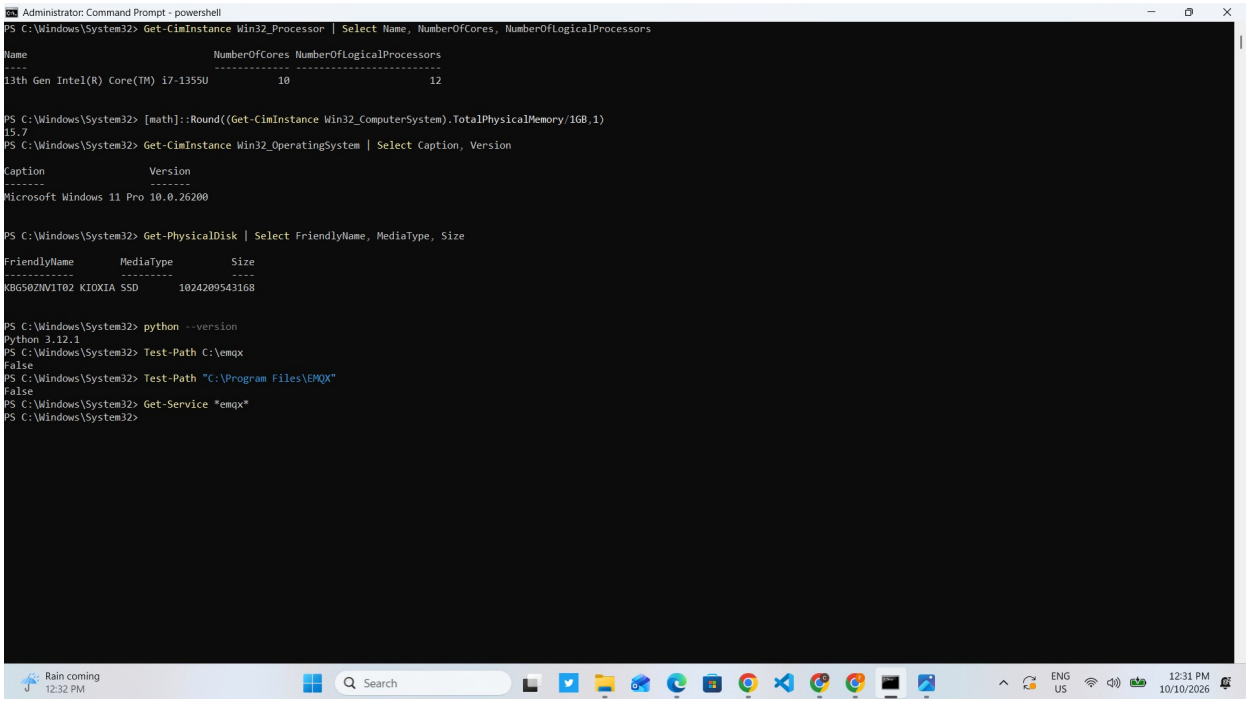


Figure 3 – CPU, RAM, operating system, storage type and Python version (10 Oct 2026, 12:31 PM).

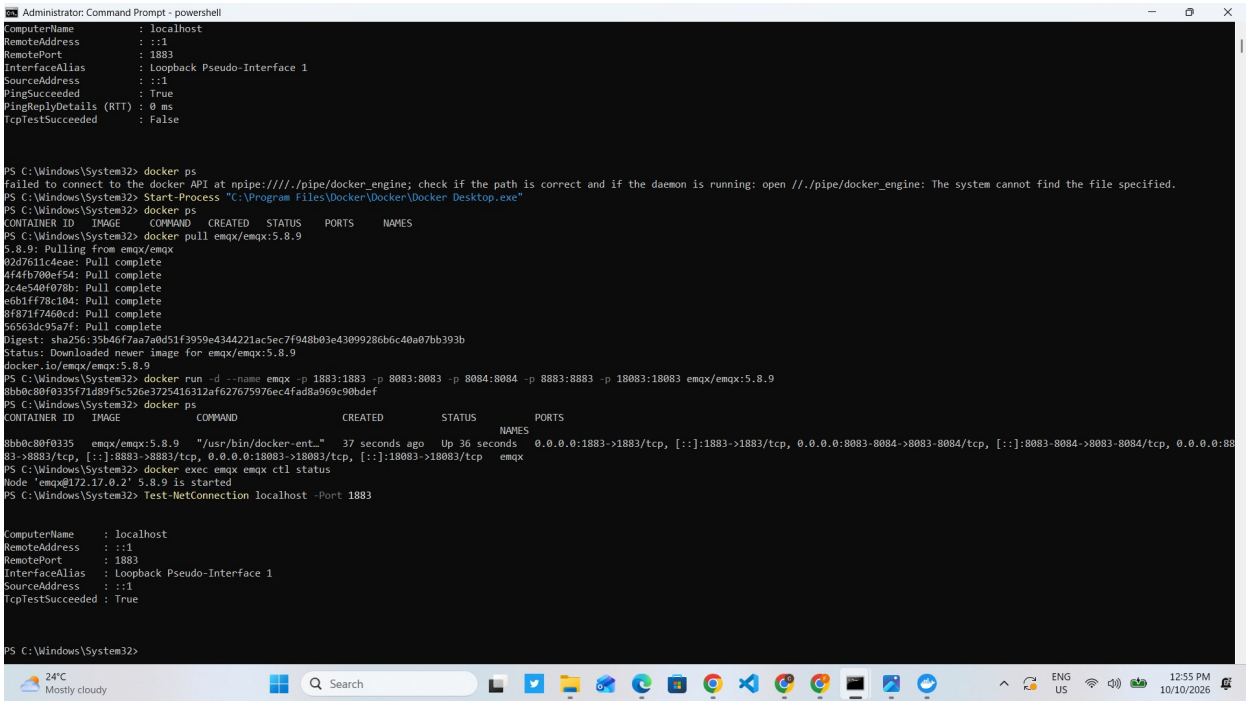


Figure 4 – EMQX started with Docker (image emqx/emqx:5.8.9): node status and TCP test of port 1883 (10 Oct 2026, 12:55 PM).

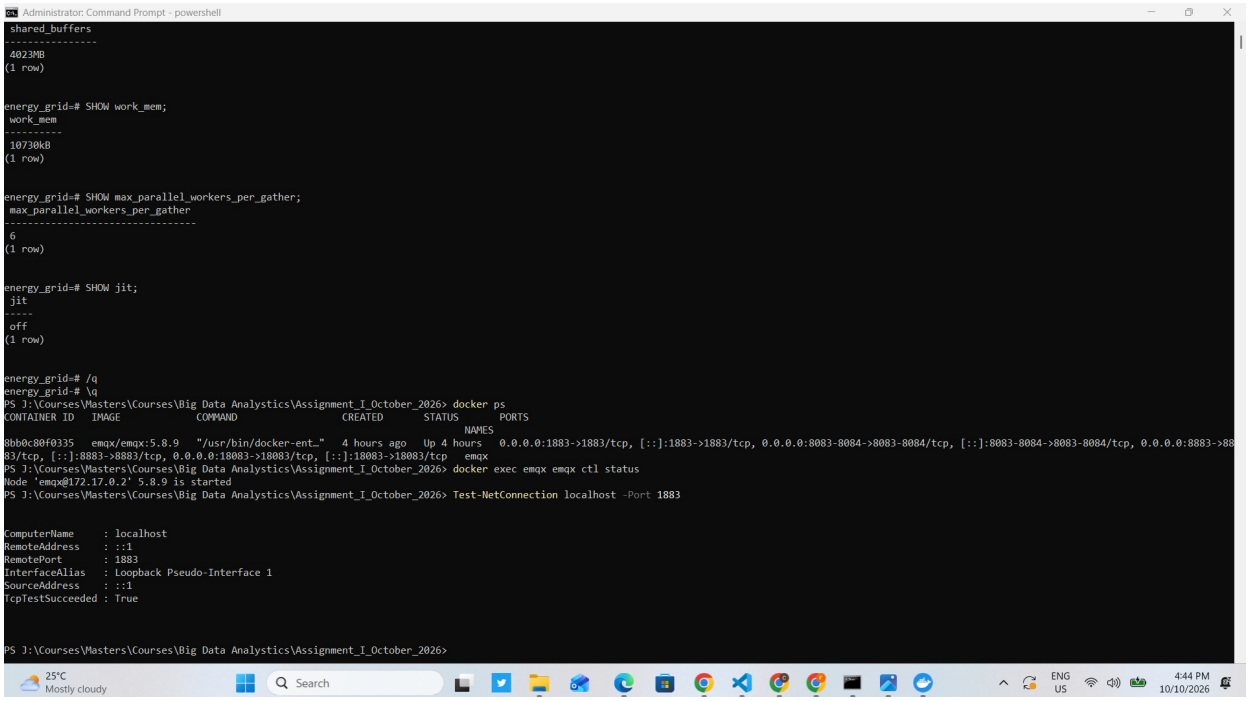


Figure 5 – Server settings (shared_buffers, work_mem, max_parallel_workers_per_gather, jit) and the EMQX container still running after four hours (10 Oct 2026, 4:44 PM).

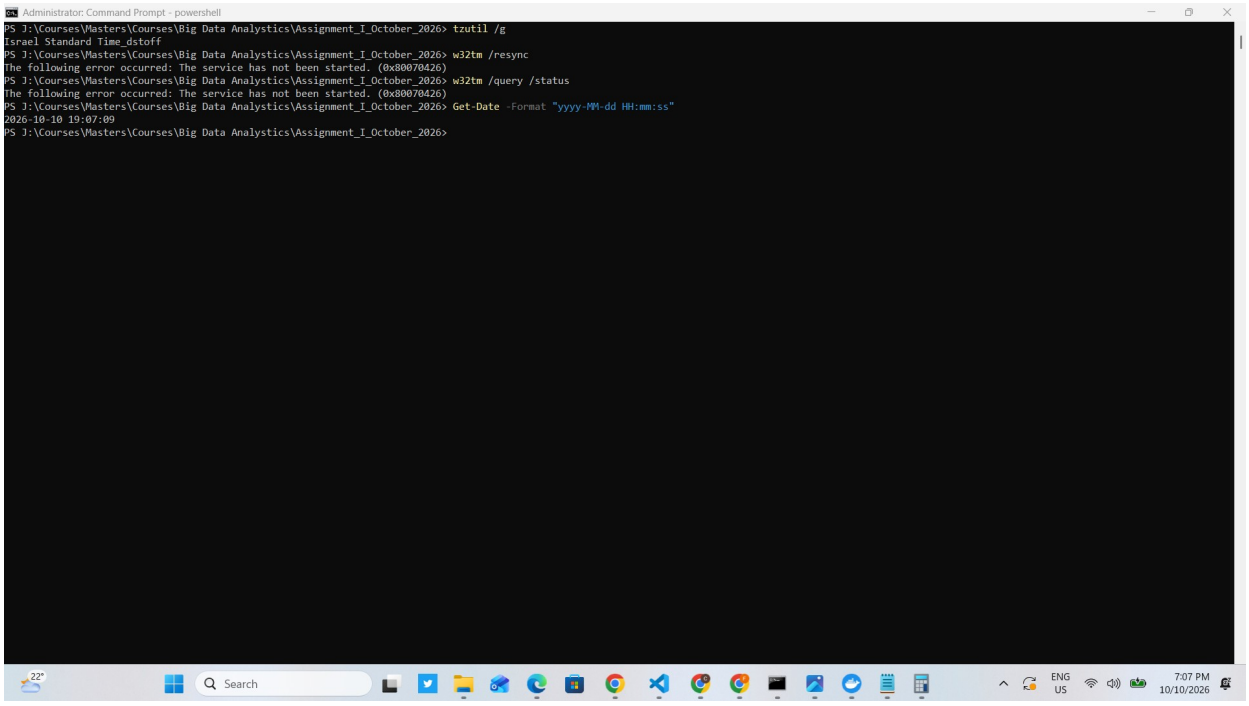


Figure 6 – Time-zone and clock check on the machine (10 Oct 2026, 7:07 PM).

1.2 Create the database structures

Three tables hold the project data: `energy_live` (regular table, pilot stream), `energy_readings` (TimescaleDB hypertable partitioned on *event_time* with one-day chunks) and `meters` (static metadata, 4,000 rows loaded with `meter_metadata()`). Both reading tables share the same eight columns and allow duplicates at ingestion time.

Data dictionary

Table	Column	PostgreSQL type	Meaning	Unit
energy_live / energy_readings	meter_id	BIGINT	Smart-meter identifier (1000000000 – 1000003999)	–
	event_time	TIMESTAMPTZ	Start of the 15-minute interval measured	timestamp (CAT)
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading (not insertion time)	timestamp (CAT)
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Voltage measurement	V
	current_a	DOUBLE PRECISION	Current measurement	A
	frequency_hz	DOUBLE PRECISION	Grid frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy over the interval (≈ power_kw × 0.25 h)	kWh
meters	meter_id	BIGINT (PK)	Meter identifier, primary key	–
	region	TEXT	Region 1 … Region 10 (400 meters each)	–
	customer_type	TEXT	residential, commercial or industrial	–
	base_power_kw	DOUBLE PRECISION	Baseline power level of the meter	kW
	power_factor	DOUBLE PRECISION	Power factor (0.95)	none

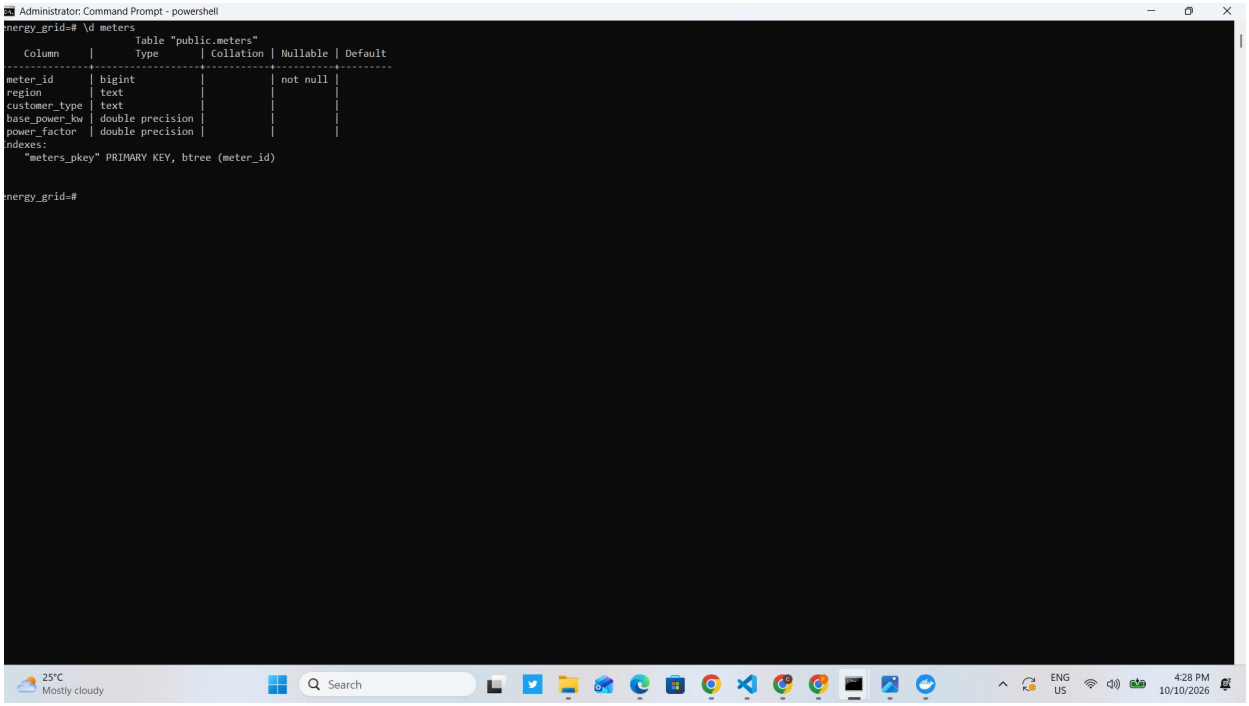


Figure 7 – Structure of table meters (\d meters): five columns, primary key meters_pkey (10 Oct 2026, 4:28 PM).

1.3 Capture and publish a pilot stream

The pilot replays the first 2,000 dictionaries from `iter_readings()` through MQTT (topic `energy/meters/{meter_id}`, QoS 1, no retained messages) with `publisher.py`. `subscriber.py`, started beforehand on `energy/meters/#` at QoS 1, parses every JSON message and inserts its eight fields into the initially empty energy_live table. Four counts are compared against the target of 2,000: dictionaries taken from the generator, messages acknowledged by the broker, messages received by the subscriber and rows committed in energy_live.

2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream

The full stream covers 4,000 meters every 15 minutes from 1 September 2026 00:00 to 29 September 2026 00:00 (exclusive): 10,752,000 planned readings, of which 10,696,848 are emitted by the generator (including duplicate copies). All emitted rows are loaded into the empty energy_readings hypertable, duplicates included.

Reference count (dataset_info)	Expected	Meaning
planned_readings	10,752,000	4,000 meters × 28 days × 96 intervals
emitted_readings	10,696,848	All yielded dictionaries, including duplicates
unique_readings	10,643,735	Distinct (meter_id, event_time) pairs
duplicate_readings	53,113	Extra copies beyond one row per pair
delayed_unique_readings	213,090	Unique pairs arriving after the interval end
missing_readings	108,265	Planned pairs with no generated reading

2.2 Investigate data quality

Using the reference counts, the quality indicators of the 28-day period are computed below. The unique-reading count 10,643,735 is confirmed independently by the three continuous aggregates (§6.1) and by the anomaly-detection run (§5.4), which both report exactly 10,643,735 readings.

Indicator	Formula	Value
Missing percentage	100 × 108,265 / 10,752,000	1.007 %
Duplicate percentage	100 × 53,113 / 10,643,735	0.499 %
Delayed percentage	100 × 213,090 / 10,643,735	2.002 %
Completeness	100 × 10,643,735 / 10,752,000	98.993 %
Check 1 – unique + duplicates = emitted	10,643,735 + 53,113	10,696,848
Check 2 – planned − unique = missing	10,752,000 − 10,643,735	108,265

2.3 Clean and preserve a consistent benchmark

Rows sharing the same (meter_id, event_time) were compared on received_time, power_kw, voltage_v, current_a, frequency_hz and energy_kwh. Because duplicate copies match in all fields, one row per pair is retained and the extra copies are deleted with repeatable code. Missing intervals are kept as gaps.

Result. The cleaned energy_readings table contains 10,643,735 rows with a total energy of 3,853,704.35 kWh, consistent with every downstream check in this report (continuous aggregates, anomaly scoring, dataset preparation).

Why missing readings must not become zero. A missing reading means no information, not no consumption. Filling gaps with zero would understate energy totals and average power, create artificial demand drops that an anomaly detector would flag, bias forecasting targets downward, and hide communication failures that the utility needs to see in its completeness indicators. Gaps are therefore preserved and reported explicitly.

3 Compare chunk intervals

3.1 Create and verify three configurations

Three hypertables hold the same cleaned data: `energy_readings` (1-day chunks), `energy_readings_3h` (3-hour chunks) and `energy_readings_week` (1-week chunks). The cleaned rows were copied into the two new tables (two statements, `INSERT 0 10643735` each, 31.2 s and 36.7 s), and each table received the same two non-unique B-tree indexes, one on event_time and one on (meter_id, event_time); the six index builds took between 5.8 s and 181.9 s, and ANALYZE was run on the three tables (10.7 s, 53.4 s and 6.7 s).

Configuration	Chunk interval	Chunks	First chunk start	Last chunk end
energy_readings	1 day	29	2026-08-31 02:00:00+02	2026-09-29 02:00:00+02
energy_readings_3h	3 hours	225	2026-08-31 23:00:00+02	2026-09-29 02:00:00+02
energy_readings_week	7 days	5	2026-08-27 02:00:00+02	2026-10-01 02:00:00+02

Table	Index	Definition
energy_readings	idx_er_event_time / idx_er_meter_time	btree (event_time) / btree (meter_id, event_time)
energy_readings_3h	idx_er3h_event_time / idx_er3h_meter_time	btree (event_time) / btree (meter_id, event_time)
energy_readings_week	idx_erweek_event_time / idx_erweek_meter_time	btree (event_time) / btree (meter_id, event_time)

Table	Rows	Meters	Earliest event_time	Latest event_time
energy_readings_week	10,643,735	4,000	2026-09-01 00:00:00+02	2026-09-28 23:45:00+02
energy_readings_3h	10,643,735	4,000	2026-09-01 00:00:00+02	2026-09-28 23:45:00+02
energy_readings	10,643,735	4,000	2026-09-01 00:00:00+02	2026-09-28 23:45:00+02

Table	sum(power_kw)	sum(energy_kwh)
energy_readings_week	15414817.387793025	3853704.3468154166
energy_readings_3h	15414817.387793003	3853704.346815412
energy_readings	15414817.387793005	3853704.3468154133

All three tables hold 10,643,735 rows and 4,000 meters with identical date limits. The sums differ by at most 2.2×10^{-8} (power) and 4.6×10^{-9} (energy), far below the 0.001 tolerance (floating-point rounding).

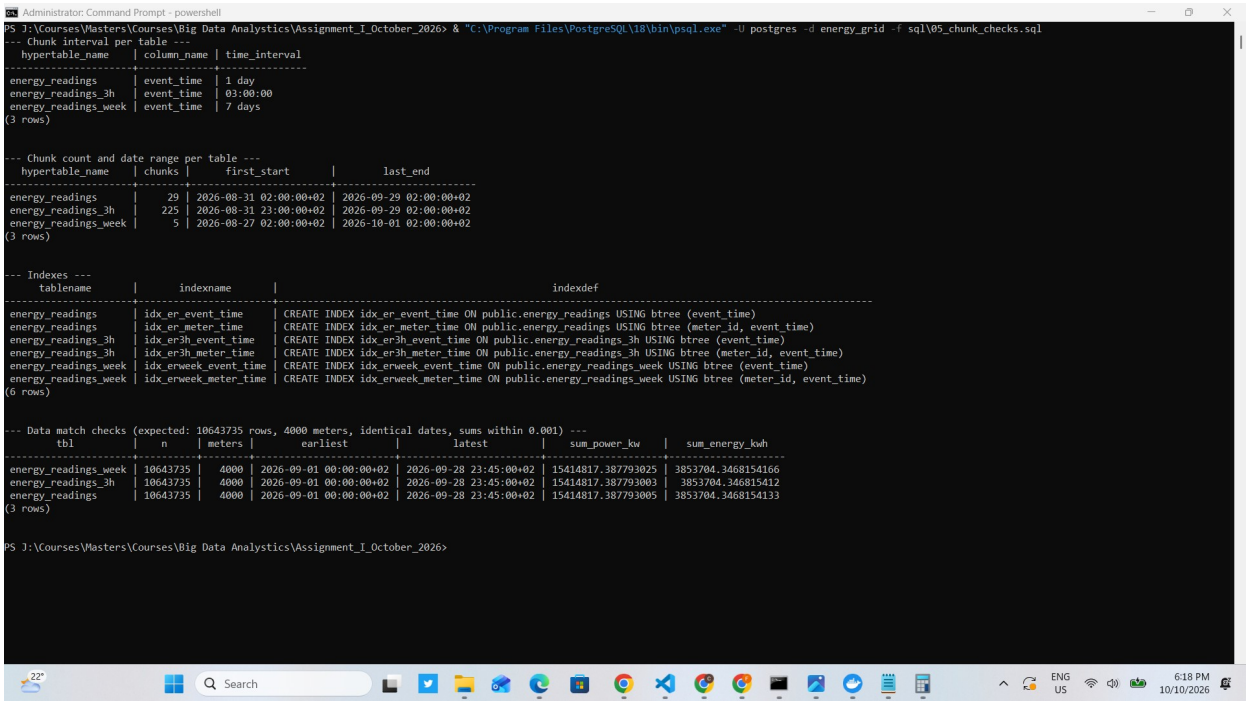


Figure 8 – Chunk interval, chunk count and date range per table, indexes and data-match checks (10 Oct 2026, 6:18 PM).

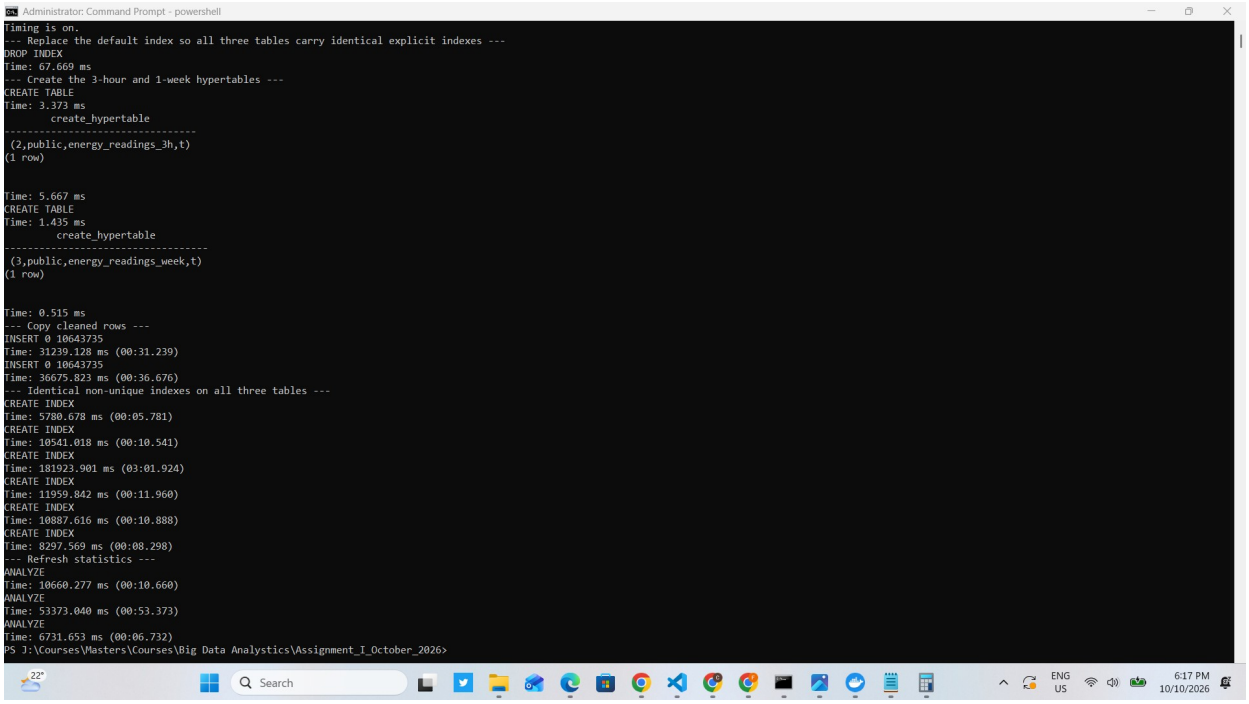


Figure 9 – Creation of the 3-hour and 1-week hypertables, copy of the cleaned rows, index creation and ANALYZE with timings (10 Oct 2026, 6:17 PM).

3.2 Implement and time the five workloads

Q1: hourly average power for 21 September; Q2: top ten 15-minute network demand intervals for 22–28 September; Q3: total energy per meter; Q4: totals and peak for meter 1000000000 (22–28 September); Q5: full-table scan aggregates. `bench_chunks.py` runs each query on the three tables, changing only the table name, with one warm-up and five measured executions (warm-cache measurements) and ANALYZE done beforehand. The table gives the median, minimum and maximum of the five measured executions in milliseconds and the session time from the first warm-up submission to the last measured result; every result matches the reference result.

Query	Table	Median (ms)	Min (ms)	Max (ms)	Session (s)	Matches reference
Q1	energy_readings	130.913	125.942	139.170	0.89	True
Q1	energy_readings_3h	116.532	112.661	144.464	0.75	True
Q1	energy_readings_week	144.221	128.562	152.683	0.89	True
Q2	energy_readings	178.151	170.073	191.341	1.30	True
Q2	energy_readings_3h	217.190	208.791	237.617	1.70	True
Q2	energy_readings_week	215.607	201.930	237.309	1.56	True
Q3	energy_readings	778.631	739.456	827.021	5.88	True
Q3	energy_readings_3h	4,604.008	4,397.155	4,658.567	34.26	True
Q3	energy_readings_week	754.802	715.392	795.251	5.50	True
Q4	energy_readings	2.887	1.763	3.894	0.17	True
Q4	energy_readings_3h	9.275	7.659	10.259	0.07	True
Q4	energy_readings_week	2.116	1.473	2.252	0.02	True
Q5	energy_readings	581.822	561.245	642.468	4.30	True
Q5	energy_readings_3h	557.321	509.114	639.862	3.51	True
Q5	energy_readings_week	598.973	579.230	653.865	3.84	True

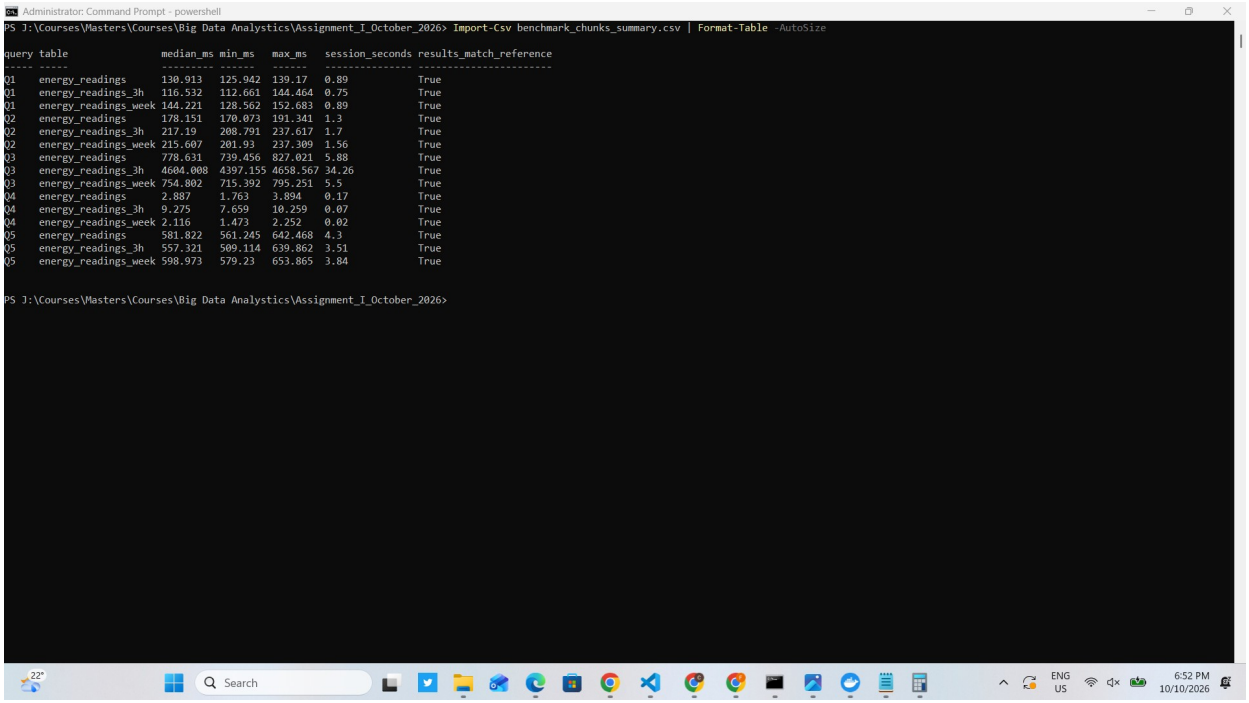


Figure 10 – Timing summary per query and table (benchmark_chunks_summary.csv) (10 Oct 2026, 6:52 PM).

3.3 Explain the chunk results

Median times and their change relative to the 1-day table (negative = faster):

Query	Fastest	1 day (ms)	3 hours (ms)	1 week (ms)	3 h vs 1 day	1 week vs 1 day
Q1	3 hours	130.9	116.5	144.2	−11.0 %	+10.2 %
Q2	1 day	178.2	217.2	215.6	+21.9 %	+21.0 %
Q3	1 week	778.6	4,604.0	754.8	+491.3 %	−3.1 %
Q4	1 week	2.9	9.3	2.1	+221.3 %	−26.7 %
Q5	3 hours	581.8	557.3	599.0	−4.2 %	+2.9 %

Chunk boundaries lie at 00:00 UTC (02:00 CAT) for the 1-day and 1-week chunks and at 21:00, 00:00, 03:00 … UTC for the 3-hour chunks. A CAT day therefore overlaps 2 daily chunks, 9 three-hour chunks and 1 weekly chunk; the 22–28 September window overlaps 8 daily, 57 three-hour and 2 weekly chunks.

Q1 – fastest: 3-hour chunks. Observation: median 116.5 ms (3 h) against 130.9 ms (1 day) and 144.2 ms (1 week); the day is 12.3 % slower than the 3-hour table and the week 23.8 % slower, but the min–max ranges of the three tables overlap. The chunks that overlap the day hold about 27 h of data (3 h chunks), 48 h (daily chunks) and 168 h (the weekly chunk). Interpretation: the order of the medians follows the amount of data in the chunks that chunk exclusion cannot discard, so finer chunks let the planner cut closer to the requested day.

Q2 – fastest: 1-day chunks. Observation: 178.2 ms against 217.2 ms (3 h, +21.9 %) and 215.6 ms (1 week, +21.0 %); the ranges do not overlap. Interpretation: the 7-day window touches 8 daily chunks, which are enough to spread over the parallel workers without much per-chunk overhead, whereas the 3-hour table must plan and merge 57 chunks and the weekly table offers only 2 chunks.

Q3 – fastest: 1-week chunks. Observation: 754.8 ms (1 week), 778.6 ms (1 day, +3.2 %) and 4,604.0 ms (3 h, 5.9× the daily table); the session time is 34.3 s for the 3-hour table against 5.5–5.9 s. Q3 reads the whole table and groups by meter, and the 3-hour table has 225 chunks against 29 and 5. Interpretation: each chunk produces its own partial aggregate of 4,000 meter groups that must then be merged, so the cost grows with the number of chunks. Q5, which scans the same rows with a single group, is not slower on the 3-hour table (557.3 ms), which points to per-chunk grouping and planning overhead and not to the scan itself.

Q4 – fastest: 1-week chunks. Observation: 2.1 ms (1 week, 2 chunks), 2.9 ms (1 day, 8 chunks) and 9.3 ms (3 h, 57 chunks): the medians follow the number of chunks touched. Interpretation: every touched chunk adds planning and an index probe on (meter_id, event_time), so a very selective query is best served by few chunks. The absolute differences are below 7.2 ms.

Q5 – no clear winner (3-hour chunks have the lowest median). Observation: 557.3 ms (3 h), 581.8 ms (1 day) and 599.0 ms (1 week); the differences (−4.2 % and +2.9 % against the daily table) are smaller than the spread of the runs (day 561.2–642.5 ms, 3 h 509.1–639.9 ms, week 579.2–653.9 ms). Interpretation: a full scan with one result group reads the same 10.6 million rows whatever the chunking.

Conclusion. No interval wins everywhere. The 1-day table is the fastest on Q2 and never more than 12.3 % slower than the best on Q1, Q3 and Q5, and 0.8 ms slower on Q4, while the 3-hour table is 5.9× slower on Q3 and 3.2× slower on Q4 and the weekly table is 21 % slower on Q2. The 1-day interval is the best compromise and is used in §4.

4 Compare compression strategies

4.1 Create the two compressed configurations

Two empty hypertables with the 1-day chunk interval chosen in §3.3 (29 chunks each) were created: `energy_time_compressed` (ordered by event_time DESC, no segmentation) and `energy_meter_compressed` (same ordering, segmented by meter_id). A compression policy for chunks older than 48 hours was added to each (job 1000 for energy_time_compressed, job 1001 for energy_meter_compressed; *compress_after* = 48:00:00, visible in Figures 26 and 29) and paused until the measurements were complete. One cutoff was recorded before loading and used for both tables: experiment start 2026-10-10 22:07:37 CAT minus 48 hours = **2026-10-08 22:07:37 CAT**. `load_compress.py` copies the cleaned data one source chunk at a time, and after each chunk is fully loaded it compresses the chunk manually if its end precedes the cutoff, timing the load and the compression separately. The data end on 28 September, so the end of every chunk precedes the cutoff and all 29 chunks of both tables are compressed.

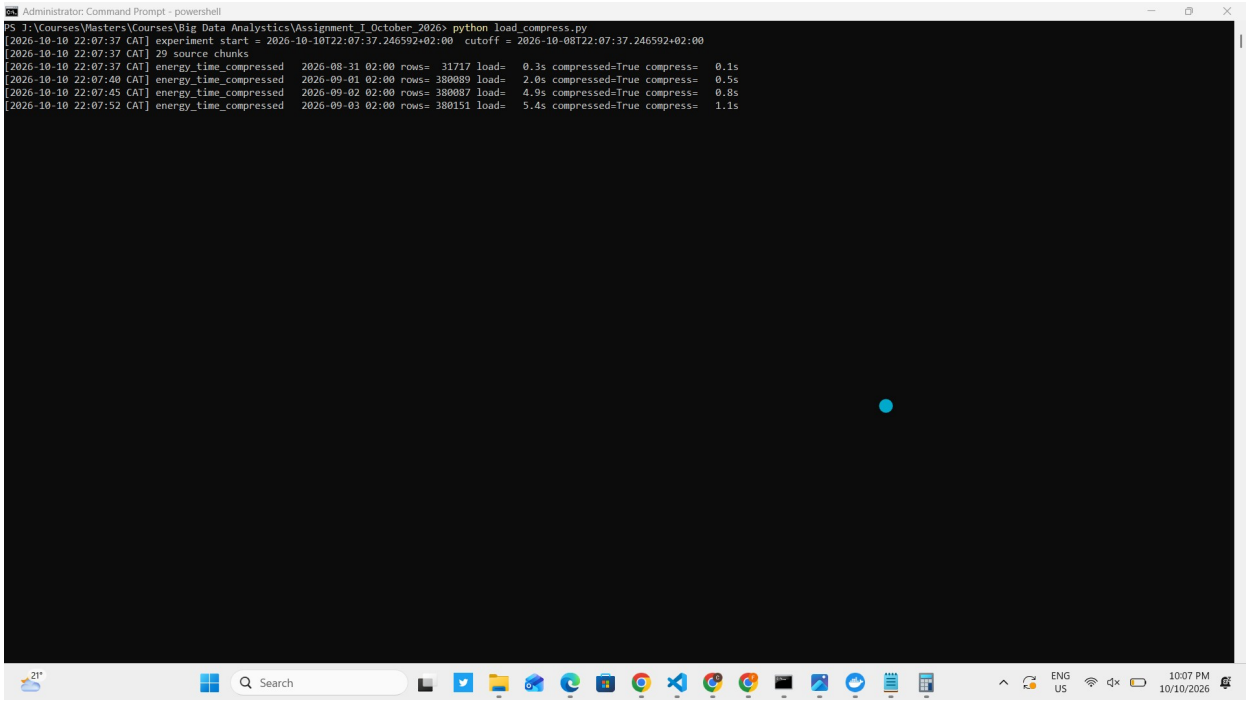


Figure 11 – Start of load_compress.py: experiment start, cutoff and the first chunks of energy_time_compressed with load and compression times (10 Oct 2026, 10:07 PM).

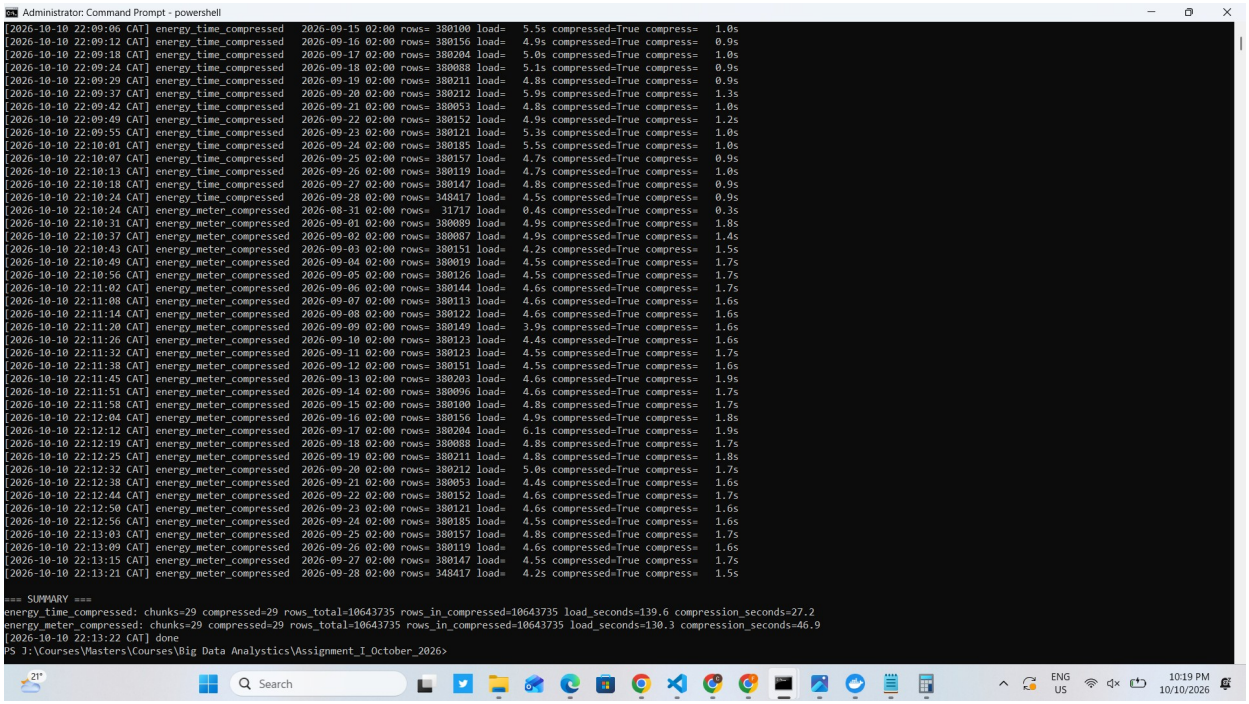


Figure 12 – Remaining chunks of both tables and the summary of load and compression times (10 Oct 2026, 10:19 PM).

4.2 Measure storage and compression cost

Sizes include all chunks and their indexes; the compressed batches are stored in the TOAST relation. Baseline: the uncompressed 1-day table energy_readings.

Table	Data	Index	TOAST	Total	Total bytes
energy_readings (baseline)	951 MB	393 MB	0 bytes	1,344 MB	1,408,974,848
energy_time_compressed	15 MB	944 kB	423 MB	439 MB	460,038,144
energy_meter_compressed	27 MB	5,584 kB	469 MB	502 MB	526,016,512

Measure	energy_time_compressed (no segmentation)	energy_meter_compressed (segment by meter_id)
Compression ratio (baseline total / compressed total)	3.06	2.68
Percentage saved	67.3 %	62.7 %
Compressed chunks / total chunks	29 / 29	29 / 29
Rows in compressed chunks	10,643,735 of 10,643,735 (100 %)	10,643,735 of 10,643,735 (100 %)
Sum of compression times (excluding loading)	27.2 s	46.9 s
Load time (excluding compression)	139.6 s	130.3 s

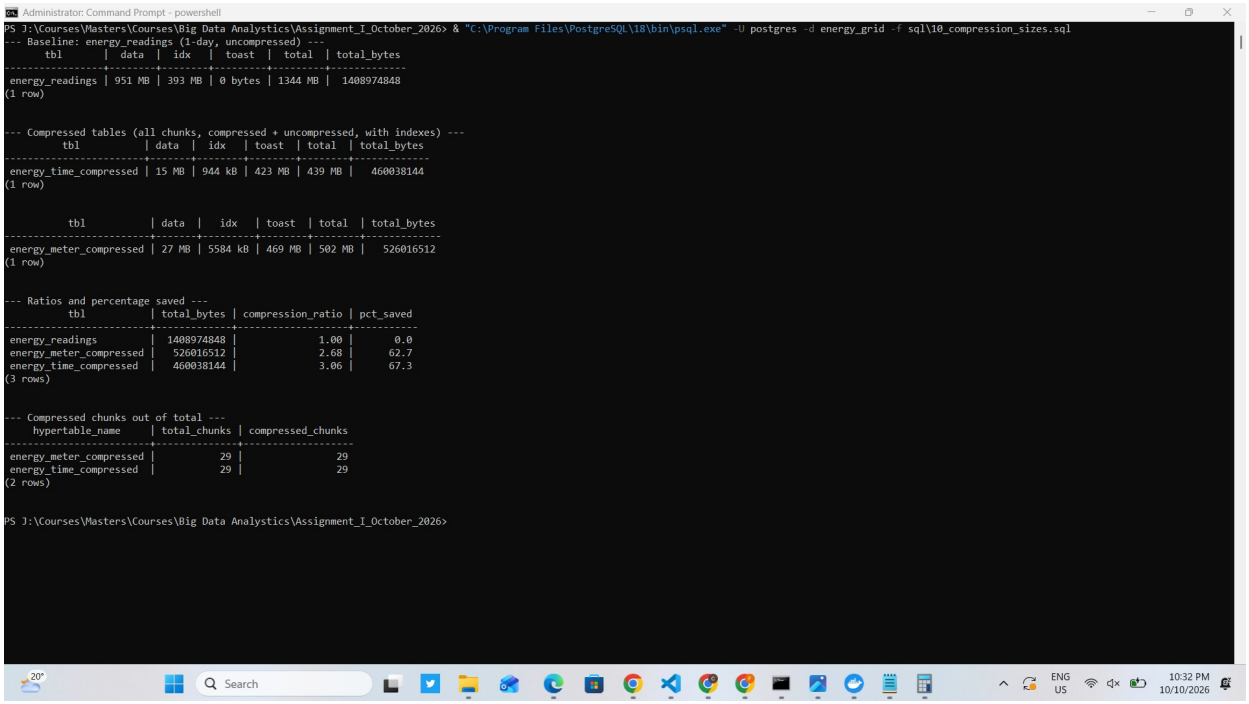


Figure 13 – Baseline and compressed sizes, compression ratios, percentage saved and compressed chunks out of total (10 Oct 2026, 10:32 PM).

Observation. Both strategies shrink the 1,344 MB baseline by more than 60 %, but segmentation by meter_id gives the larger table (502 MB against 439 MB, +14.3 %) and the longer compression (46.9 s against 27.2 s, +72 %; about 1.6–1.9 s per full-size chunk against about 1 s, Figure 12). The indexes shrink from 393 MB to 944 kB and 5,584 kB. **Interpretation.** With one-day chunks each meter has at most 96 rows per chunk, so the segmented batches are small and carry more metadata, which lowers the ratio; ordering and grouping the rows of 4,000 meters before compressing is also more work per chunk.

4.3 Repeat the query comparison

Both plans below are Q4 (single meter 1000000000, 22–28 September) on the two compressed tables, once with the meter as a regular column and once as the segment-by column.

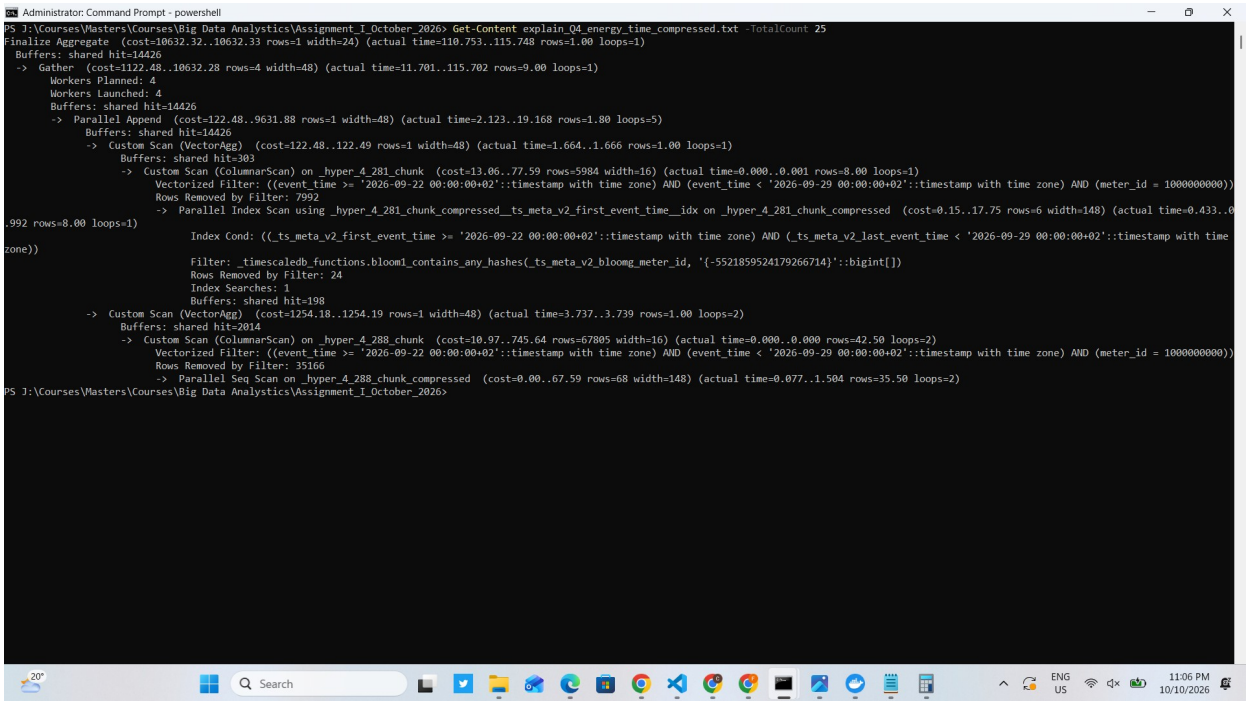


Figure 14 – Q4 execution plan on energy_time_compressed (EXPLAIN ANALYZE, BUFFERS) – 10 Oct 2026, 11:06 PM.

Plan without segmentation (energy_time_compressed)

- The plan uses a **Parallel Append** over two chunks with **Custom Scan (VectorAgg)** on top of **Custom Scan (ColumnarScan)**: aggregation is vectorised and decompression is done on the fly (4 workers launched; actual total time ≈ 115.7 ms; 14,426 shared buffer hits, no reads).
- In the first chunk the vectorised filter on meter_id = 1000000000 is applied *after* reading batches: 7,992 rows are removed by the filter and only 8 are kept. On the compressed-batch metadata a **bloom filter** (bloom1_contains_any_hashes on meter_id) removes 24 of the candidate batches before decompression.
- In the second chunk, a parallel sequential scan on the compressed chunk reads all batches (35,166 rows removed by the filter, ≈ 42 rows kept per loop): there is no segment-level index that jumps directly to the meter.

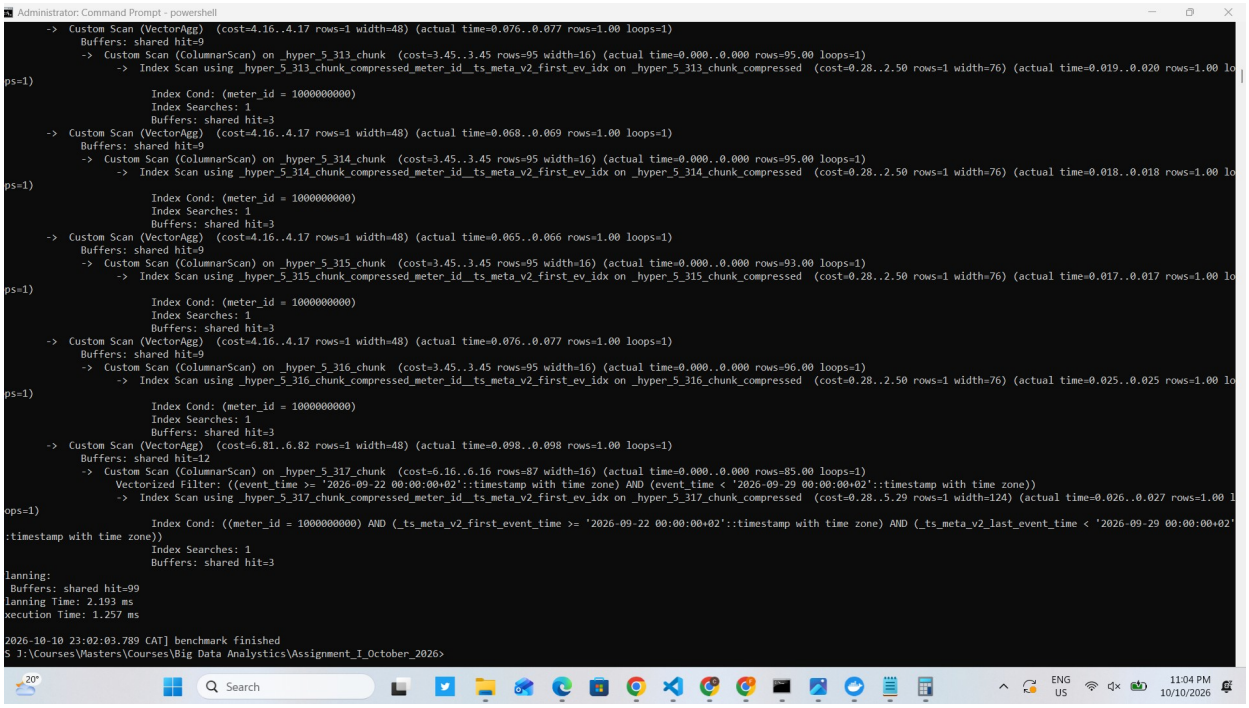


Figure 15 – Q4 execution plan on energy_meter_compressed (EXPLAIN ANALYZE, BUFFERS) (10 Oct 2026, 11:04 PM).

Plan with segmentation (energy_meter_compressed)

- For every compressed chunk the plan is a **Custom Scan (VectorAgg)** over a **Custom Scan (ColumnarScan)** fed by an **Index Scan** on the compressed chunk with *Index Cond: (meter_id = 1000000000)* and a single index search; each chunk takes about 0.02 ms and 3 shared buffer hits.
- Whole query: 99 shared buffer hits, planning time 2.193 ms and execution time 1.257 ms.

Comparison. Observation: in these two EXPLAIN ANALYZE runs the same query takes ≈ 115.7 ms with 14,426 buffer hits on the unsegmented table and 1.257 ms with 99 buffer hits on the segmented table, i.e. ≈ 92× less time and ≈ 146× fewer buffer hits (single executions, not medians; for reference, the median of five executions on the uncompressed 1-day table is 2.887 ms, §3.2). Interpretation: without segmentation the rows of all 4,000 meters are interleaved in each compressed batch, so the engine must filter many batches (the bloom filter only reduces this work); with segmentation it jumps directly to the batches of the requested meter.

4.4 Select a storage configuration

Effect of meter segmentation. Segmentation turns the single-meter workload Q4 from a scan of all batches (≈ 115.7 ms, 14,426 buffer hits) into an index lookup of the meter's own batches (1.257 ms, 99 buffer hits). A query over all meters (Q1–Q3, Q5) reads every batch in both layouts, so segmentation gives it no selective access path (interpretation); the price of segmentation is paid by all workloads in storage and compression time.

Recommendation. Use **1-day chunks** with the **compression segmented by meter_id** (energy_meter_compressed). The 1-day interval is the best compromise of §3.3 (fastest on Q2, at most 12.3 % behind the best on Q1, Q3 and Q5, 0.8 ms behind on Q4, and free of the 5.9× slowdown of 3-hour chunks on Q3). Compression cuts the 1,344 MB baseline to 502 MB (ratio 2.68, 62.7 % saved; 439 MB and 3.06 without segmentation), and the segmented layout answers the single-meter query Q4 in 1.257 ms instead of ≈ 115.7 ms. Network-wide reports are served by the continuous aggregates (§6–§7), so the compressed raw table is expected to serve mainly single-meter and ad-hoc queries.

Trade-off. The segmented layout needs 63 MB (14.3 %) more storage than the unsegmented one and 72 % more compression time (46.9 s against 27.2 s for the 29 chunks, paid again for every chunk the policy compresses). If almost all queries scan every meter, the unsegmented layout is the cheaper choice.

5 Investigate the utility data

5.1 Identify demand patterns

The analysis program sums energy_kwh per CAT day and per seven-day period (1, 8, 15 and 22 September) and sums power_kw across meters at each event_time to find the highest 15-minute demand. The figure shows (top-left) network energy per CAT day, (top-right) totals of the four seven-day periods, (bottom-left) the average network demand by CAT hour of day and (bottom-right) the network demand per 15-minute interval with the three highest peaks in red.

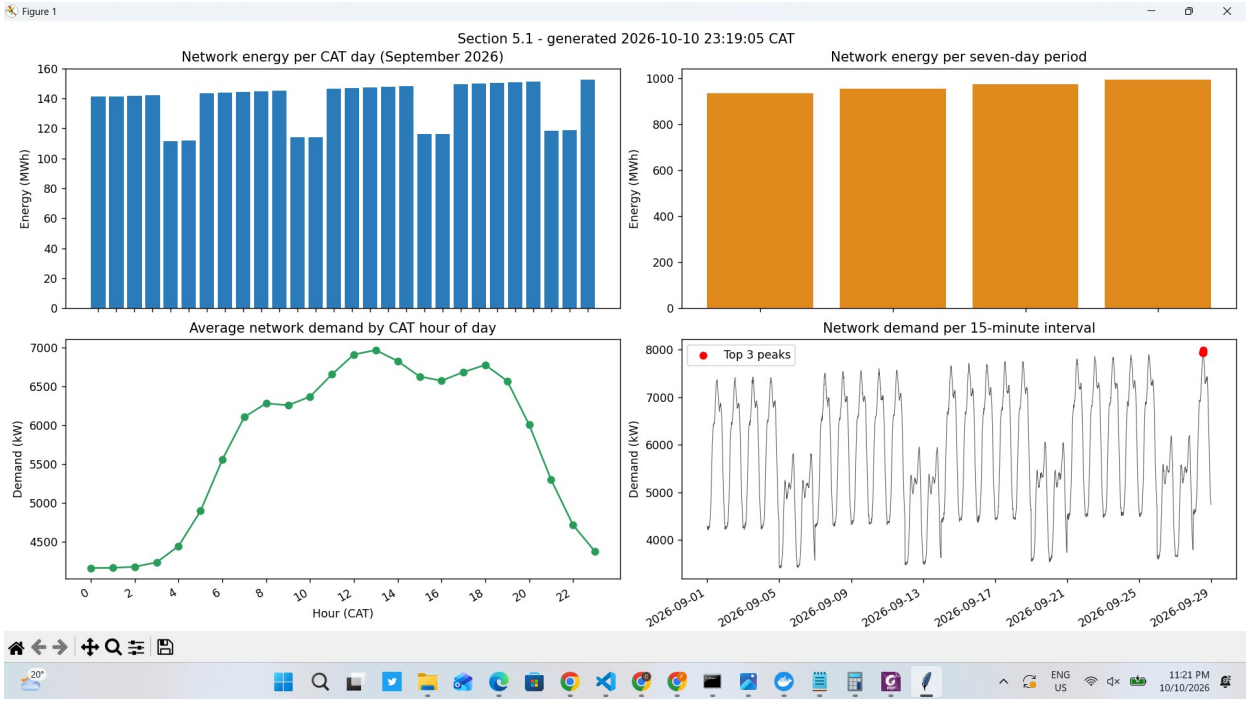


Figure 16 – Section 5.1 – daily and weekly energy, hourly demand profile and 15-minute demand (10 Oct 2026, 11:21 PM).

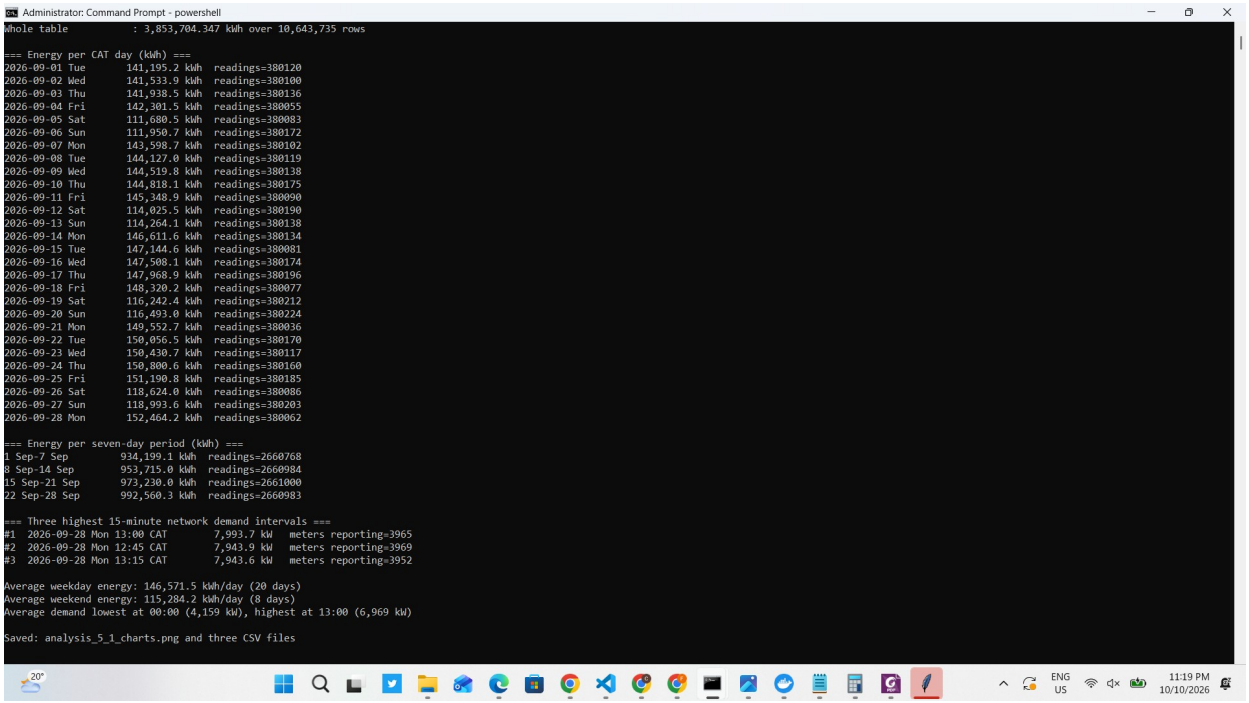


Figure 17 – Section 5.1 – numerical results: energy per CAT day, energy per seven-day period and the three highest 15-minute intervals (10 Oct 2026, 11:19 PM).

Three highest 15-minute network demand intervals (sum of power_kw across meters):

Rank	Interval start (CAT)	Network demand (kW)	Meters reporting
#1	2026-09-28 (Mon) 13:00	7,993.7	3,965
#2	2026-09-28 (Mon) 12:45	7,943.9	3,969
#3	2026-09-28 (Mon) 13:15	7,943.6	3,952

Observations

- **Weekly rhythm.** The 20 weekdays use 141.2–152.5 MWh per day (average 146,571.5 kWh/day), whereas the four weekend pairs (5–6, 12–13, 19–20 and 26–27 September) use 111.7–119.0 MWh (average 115,284.2 kWh/day over 8 days), i.e. 21.3 % less.
- **Growth.** The seven-day totals rise steadily from 934,199.1 kWh (1–7 September) to 953,715.0, 973,230.0 and 992,560.3 kWh (22–28 September): +6.2 % in three weeks, about +2 % per week. The highest day is Monday 28 September (152,464.2 kWh).
- **Daily profile.** The average demand is lowest at 00:00 (4,159 kW), ramps up between 04:00 and 07:00, is highest at 13:00 (6,969 kW) and shows a secondary evening peak near 18:00 (≈ 6,780 kW, read from the chart) before falling after 19:00.
- **Highest 15-minute intervals.** The three highest intervals (7,993.7, 7,943.9 and 7,943.6 kW) all fall around 13:00 on Monday 28 September, the highest-energy day. Only 3,952–3,969 of the 4,000 meters reported in these intervals, so the true network peak is slightly higher than the measured one.

Operational implications and kW versus kWh

Peaks drive the capacity that must be provisioned (generation, transformers, feeders), whereas energy drives fuel and purchasing cost. The utility should schedule maintenance and flexible loads on weekends and night hours, prepare reserve capacity for weekday midday and evening peaks, and monitor the upward trend visible in the weekly totals and in the 28 September peaks. **kW** is a rate of power at an instant (here averaged over 15 minutes); **kWh** is the energy accumulated over time (kW × hours). A short high-kW peak can be costly for the grid even if it adds little kWh, and a long moderate load can add many kWh without stressing capacity.

5.2 Compare regions and customer types

`analysis_5_2.py` joins the cleaned readings to meters on meter_id and compares the totals over all 28 days by region and by customer type, divided by the number of registered meters of each group (network: 4,000 meters, 3,853,704.3 kWh, 963.4 kWh per meter, 10,643,735 readings).

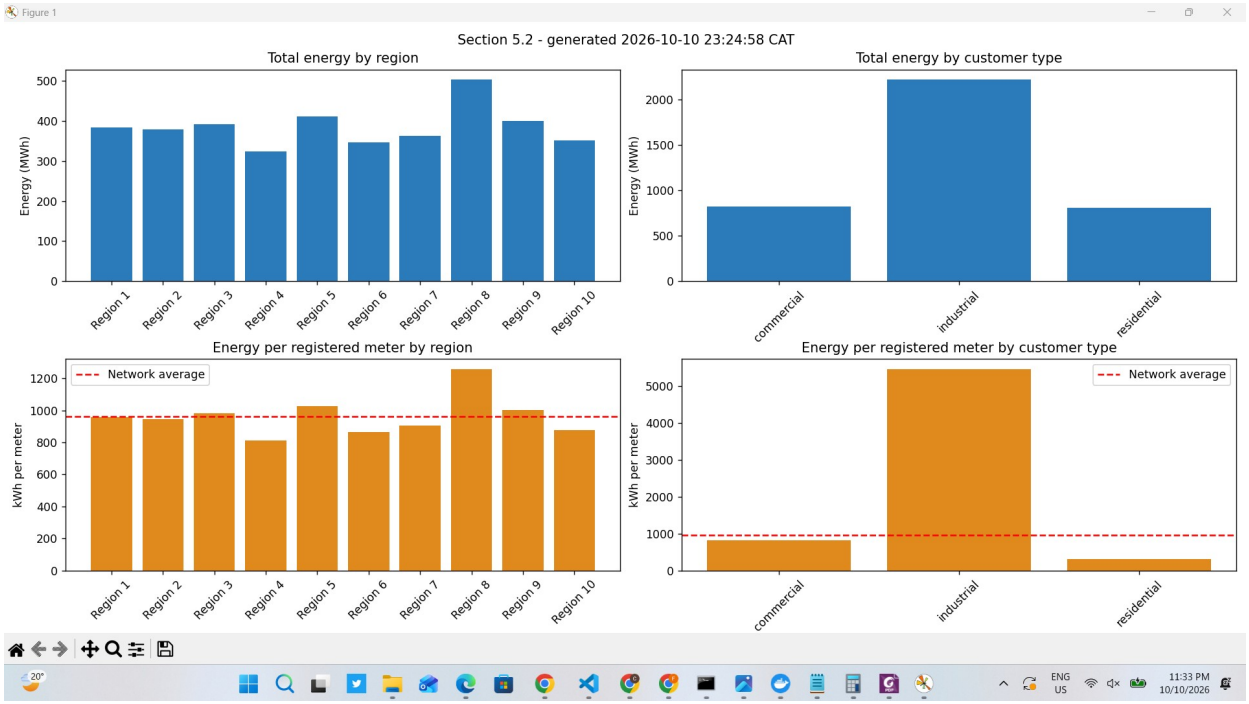


Figure 18 – Section 5.2 – total energy and energy per registered meter by region and customer type (10 Oct 2026, 11:33 PM).

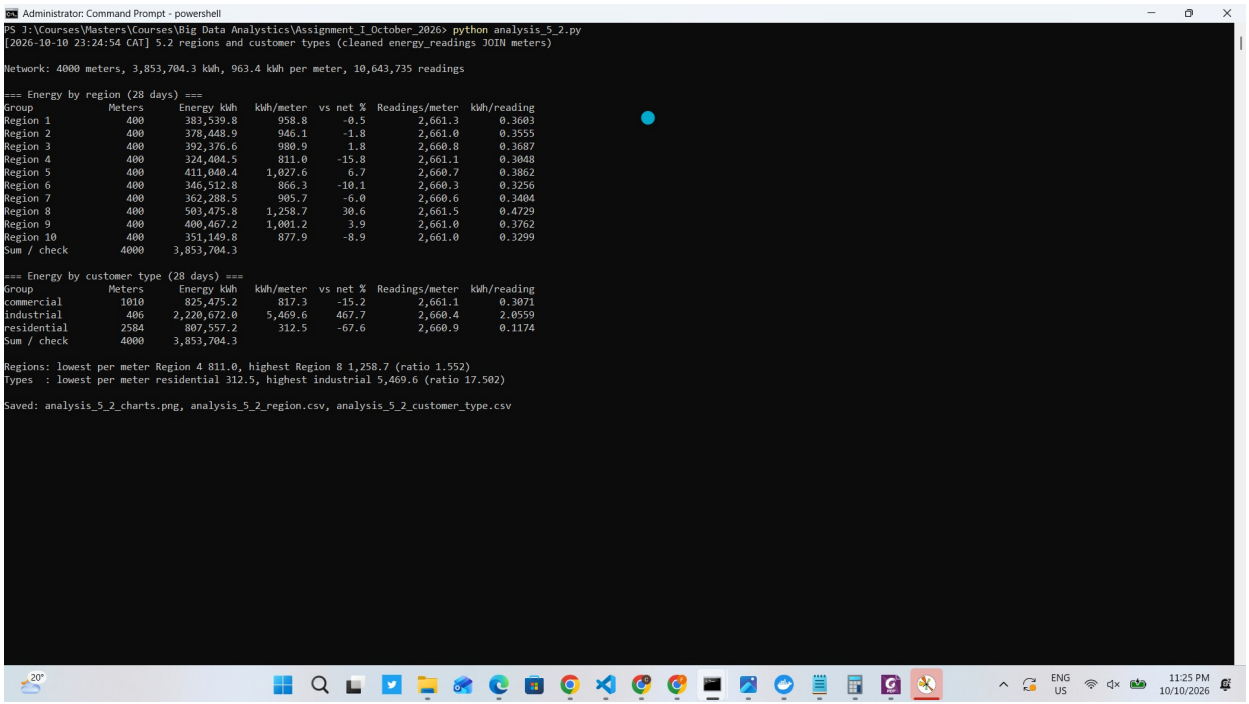


Figure 19 – Section 5.2 – group counts, totals and per-meter values by region and customer type (10 Oct 2026, 11:25 PM).

Region	Meters	Energy (kWh)	kWh per meter	vs network average
Region 1	400	383,539.8	958.8	−0.5 %
Region 2	400	378,448.9	946.1	−1.8 %
Region 3	400	392,376.6	980.9	+1.8 %
Region 4	400	324,404.5	811.0	−15.8 %
Region 5	400	411,040.4	1,027.6	+6.7 %
Region 6	400	346,512.8	866.3	−10.1 %
Region 7	400	362,288.5	905.7	−6.0 %

Region	Meters	Energy (kWh)	kWh per meter	vs network average
Region 8	400	503,475.8	1,258.7	+30.6 %
Region 9	400	400,467.2	1,001.2	+3.9 %
Region 10	400	351,149.8	877.9	−8.9 %
Total	**4,000**	**3,853,704.3**	**963.4**	

Customer type	Meters	Energy (kWh)	kWh per meter	vs network average
commercial	1,010	825,475.2	817.3	−15.2 %
industrial	406	2,220,672.0	5,469.6	+467.7 %
residential	2,584	807,557.2	312.5	−67.6 %
Total	**4,000**	**3,853,704.3**	**963.4**	

To explain the regional differences, the meters of each region were counted by customer type and the energy per meter was computed within each region and type:

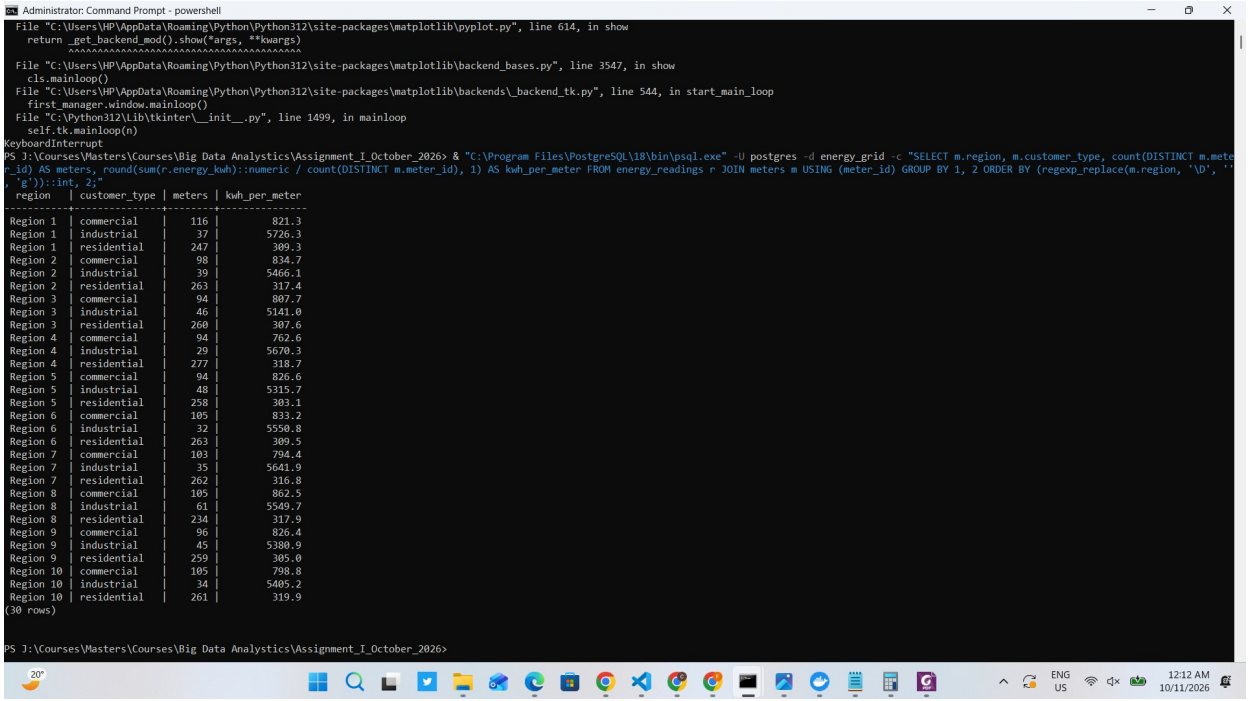


Figure 20 – Meters and kWh per meter by region and customer type (SQL query on energy_readings JOIN meters) (11 Oct 2026, 12:12 AM).

Region	Industrial meters	Industrial share	kWh per meter
Region 1	37	9.25 %	958.8
Region 2	39	9.75 %	946.1
Region 3	46	11.50 %	980.9
Region 4	29	7.25 %	811.0
Region 5	48	12.00 %	1,027.6
Region 6	32	8.00 %	866.3
Region 7	35	8.75 %	905.7
Region 8	61	15.25 %	1,258.7
Region 9	45	11.25 %	1,001.2
Region 10	34	8.50 %	877.9

What remains after normalisation, and implications. Observation: dividing by the meter count does not remove the regional differences. Region 8 stays 30.6 % above the network average (1,258.7 kWh per meter) and Region 4 15.8 % below (811.0), a ratio of 1.55 between the highest and the lowest region, while the customer types differ by a factor of 17.5 (industrial 5,469.6 against residential 312.5 kWh per meter). The number of readings per meter is almost identical in every group (2,660.3–2,661.5), so the differences are not caused by missing data. Within each customer type the energy per meter is similar across regions (industrial 5,141–5,726, commercial 763–863, residential 303–320 kWh), whereas the number of industrial meters varies from 29 (Region 4) to 61 (Region 8). The industrial share of a region and its energy per meter are strongly correlated (Pearson r = 0.97 over the ten regions). Interpretation: the regional differences come mainly from the regional customer mix, and customer type is the dominant source of heterogeneity. Planning, tariffs and demand-response programmes should therefore be segmented by customer type, regional load growth should be monitored through the industrial share of each region, and Regions 8 and 5 (the highest energy per meter, with 61 and 48 industrial meters) deserve priority for capacity checks.

5.3 Assess reporting quality

`analysis_5_3.py` computes completeness as observed unique (meter, event_time) pairs divided by the planned pairs (registered meters × 28 days × 96 readings = 1,075,200 per region and 10,752,000 for the network), and compares the energy of 21 September (event_time) counting only readings received by 00:00 on 22 September with the energy of all arrivals.

Scope	Meters	Planned	Observed	Missing	Completeness (%)
Region 1	400	1,075,200	1,064,513	10,687	99.0060
Region 2	400	1,075,200	1,064,419	10,781	98.9973
Region 3	400	1,075,200	1,064,327	10,873	98.9887
Region 4	400	1,075,200	1,064,441	10,759	98.9993
Region 5	400	1,075,200	1,064,290	10,910	98.9853
Region 6	400	1,075,200	1,064,118	11,082	98.9693
Region 7	400	1,075,200	1,064,237	10,963	98.9804
Region 8	400	1,075,200	1,064,587	10,613	99.0129
Region 9	400	1,075,200	1,064,388	10,812	98.9944
Region 10	400	1,075,200	1,064,415	10,785	98.9969
Network	**4,000**	**10,752,000**	**10,643,735**	**108,265**	**98.9931**

The network completeness over the 28 days is 98.9931 % (108,265 readings missing), matching the reference counts of §2.2; the regions range from 98.9693 % (Region 6) to 99.0129 % (Region 8). For the 22–28 September window the dashboards (§7) report region completeness between 98.96 % (Region 6) and 99.03 % (Region 4).

Energy of event_time on 21 September (CAT)	Energy (kWh)	Readings
Received by 00:00 on 22 September	149,400.545	379,505
All arrivals (final)	149,552.723	380,036
Difference	**152.178 kWh = 0.1018 % of the final total**	531 not yet received

Of the 384,000 readings planned for 21 September, 3,964 (1.032 %) were never generated. The 531 readings that had not yet arrived at midnight all belong to the last three event hours: 46 readings (11.585 kWh) of hour 21, 186 (66.474 kWh) of hour 22 and 299 (74.119 kWh) of hour 23.

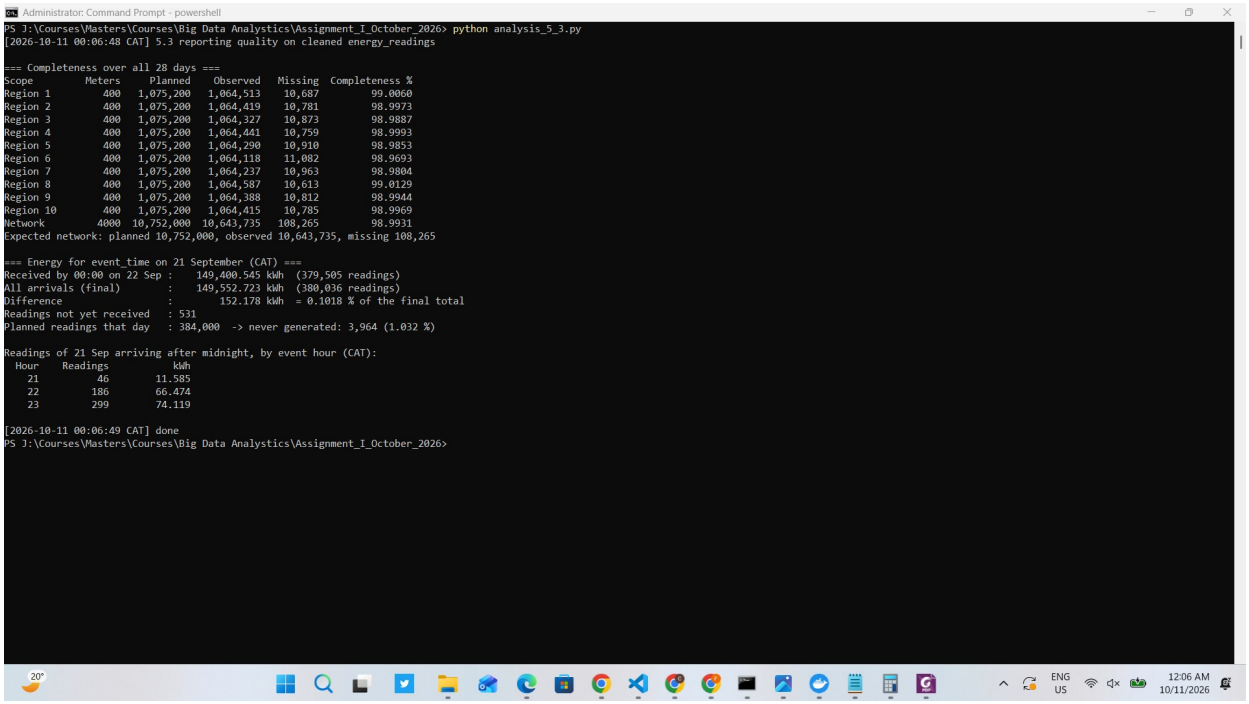


Figure 21 – Section 5.3 – completeness per region over 28 days and the two daily totals of 21 September (11 Oct 2026, 12:06 AM).

Effect of missing and delayed readings. Observation: the delayed readings change the energy total of 21 September by only 0.10 % (152 kWh), and they all concern the last hours of the day, because a delay of up to 180 minutes moves the arrival after midnight only for readings whose interval starts after 21:00. The readings that were never generated (1.03 % of the planned readings of that day, about 1.0 % of its energy if they had average consumption – an estimate) are lost for good and lower totals and completeness permanently. Interpretation: a report closed at midnight understates the last hours of the day until the late arrivals are received, so near-real-time reports must be recomputed after the 180-minute late window and must show completeness next to the energy values; missing readings must be reported as missing and not replaced by zero.

5.4 Investigate abnormal consumption

Detection rule and threshold choice

Every cleaned reading is scored against its own meter baseline: **ratio = power_kw / meter baseline**. A reading is flagged as a **spike** if ratio > 1.8 and as a **drop** if ratio < 0.4; a baseline requires at least 5 observations. The rule uses relative rather than absolute levels so that residential, commercial and industrial meters, whose loads differ by an order of magnitude, share the same thresholds. The thresholds were chosen from the distribution of ratios (histogram below): normal readings stay within 0.728 – 1.364, so 1.8 and 0.4 sit well outside natural variation while remaining sensitive to a doubling or a collapse of consumption. The labels from `anomaly_events()` were used only afterwards, for evaluation.

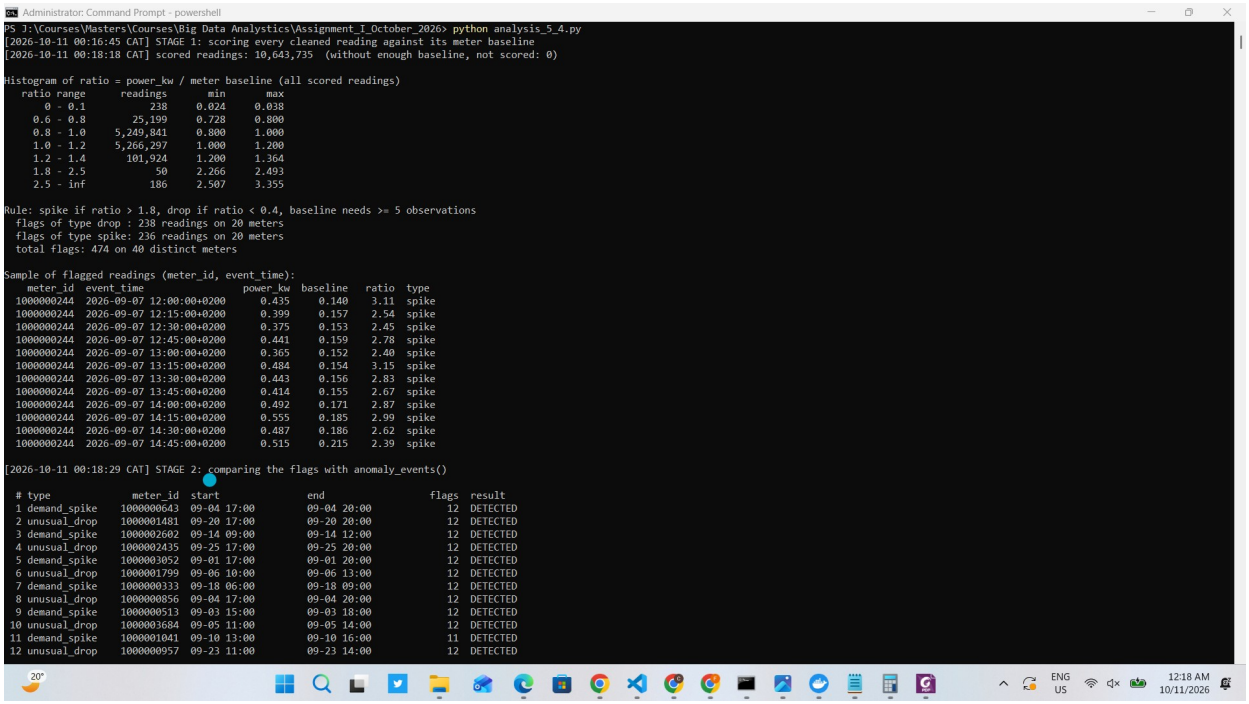


Figure 22 – Anomaly rule, ratio histogram and sample of flagged readings for meter 1000000244 (11 Oct 2026, 12:18 AM).

Ratio range	Readings	Min	Max
0 – 0.1	238	0.024	0.038
0.6 – 0.8	25,199	0.728	0.800
0.8 – 1.0	5,249,841	0.800	1.000
1.0 – 1.2	5,266,297	1.000	1.200
1.2 – 1.4	101,924	1.200	1.364
1.8 – 2.5	50	2.266	2.493
2.5 – inf	186	2.507	3.355

The histogram shows a clean separation: no reading falls between 1.364 and 2.266 or between 0.038 and 0.728. 10,643,735 readings were scored (0 without enough baseline); the rule flagged **474 readings on 40 distinct meters** – 236 spikes (20 meters) and 238 drops (20 meters).

Comparison with the 40 injected events

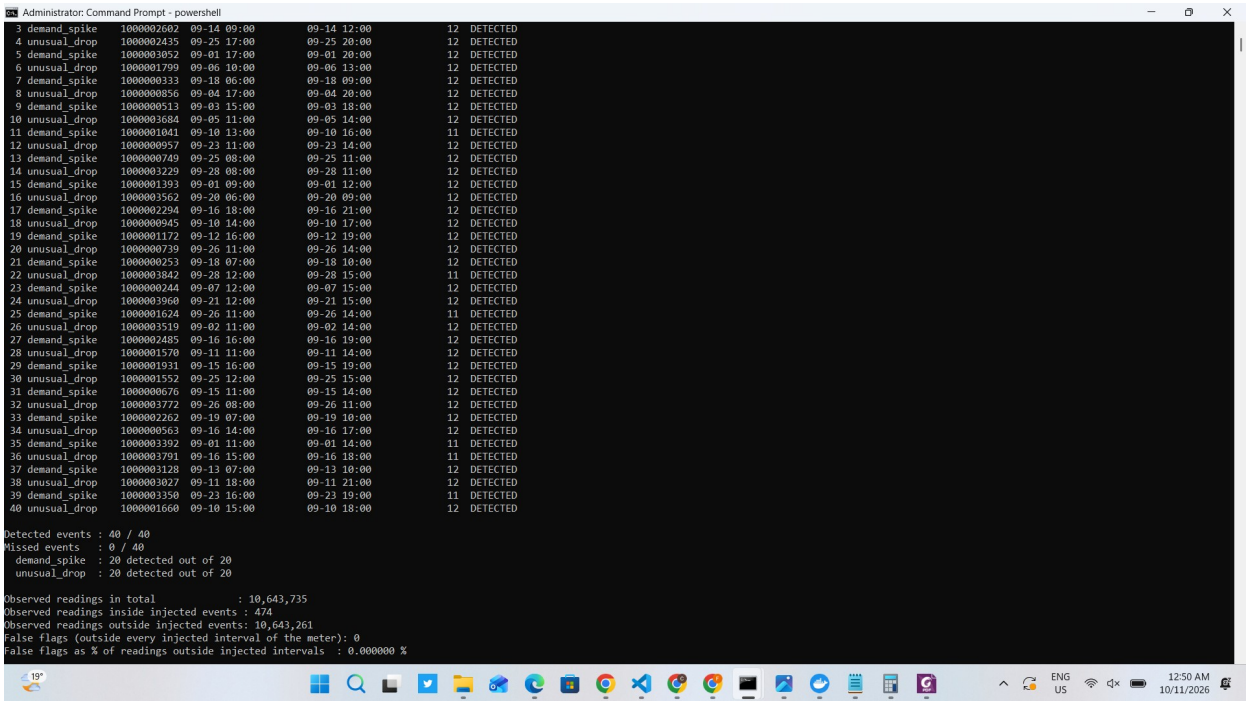


Figure 23 – Event-by-event comparison with anomaly_events() and evaluation summary (11 Oct 2026, 12:50 AM).

Evaluation measure	Result
Injected events detected	**40 / 40** (20 / 20 demand_spike, 20 / 20 unusual_drop)
Missed events	0 / 40
Observed readings inside / outside injected intervals	474 / 10,643,261 (total 10,643,735)
False flags (outside every injected interval of the meter)	0 – i.e. 0.000000 % of readings outside injected intervals
Percentage of meters affected by false flags	0 / 4,000 × 100 = 0.00 %

Missed events and false alarms. No event was missed, and each event received 11 or 12 flags out of its 12 quarter-hours; the events with 11 flags (e.g. #11, #22, #25, #35, #36, #39) most likely correspond to a reading absent from the stream (missing data), which cannot be flagged. No false alarm occurred because the injected anomalies are large and the normal ratio range is narrow. This result should be interpreted with caution: it reflects the synthetic nature of the data. Real meters have gradual drifts, weather- and holiday-driven changes and small anomalies close to the normal range, where a fixed ratio rule would trade missed events against false alarms, and where baselines that are seasonally and hourly aware would be needed.

5.5 Management recommendation (≤ 500 words)

Operational actions. (1) Plan generation and feeder capacity around weekday midday (6,969 kW average at 13:00) and the evening peak; demand peaked at 7,993.7 kW on Monday 28 September at 13:00 with 35 meters not reporting, and weekly energy rose 6.2 % over the month (934.2 → 992.6 MWh). (2) Schedule maintenance and flexible loads on weekends and at night: weekend days use 21.3 % less energy and demand is lowest at 00:00 (4,159 kW). (3) Target industrial customers (406 meters, 10 % of the meters, 58 % of the energy), which use 17.5 times more energy per meter than residential ones, for demand-response and tariff programmes, and prioritise Regions 8 and 5 (1,258.7 and 1,027.6 kWh per meter, with the most industrial meters). (4) Deploy the ratio-based anomaly rule (spike > 1.8, drop < 0.4): it detected 40 / 40 injected events with 0 false flags on 10.6 million readings. (5) Review the 1.0 % missing and 2.0 % delayed readings with the communications team and publish completeness next to every energy figure: late arrivals changed the 21 September total by only 0.10 %, but missing readings are lost.

Storage and reporting configuration. Keep the cleaned readings in a one-day TimescaleDB hypertable and serve routine reports from the three continuous aggregates (15-minute/region, hour/region, day/meter). They occupy only ≈ 18 MB and reproduce the raw totals exactly (10,643,735 readings; 3,853,704.35 kWh). Over ten measured refreshes the whole Aggregate dashboard took a median 610.8 ms against 2,780.3 ms for the Raw dashboard (≈ 4.6× faster; query time of the four all-meter panels 80–103× lower). Compress the one-day chunks segmented by meter_id: 1,344 MB shrink to 502 MB (ratio 2.68) and a single-meter query runs in 1.26 ms instead of ≈ 116 ms, at the price of 14 % more storage and 72 % more compression time than the unsegmented layout. Because real-time aggregation is disabled, figures are only as fresh as the last refresh: keep the 15-minute refresh policies, re-run them after the 180-minute late-arrival window, and use the raw table for other grains.

Forecast model. Use the gradient-boosting model (configuration C3) for day-ahead regional planning. On the 1,680 test predictions it cut MAE from 42.708 kWh (previous-day baseline) to 7.489 kWh (−82.5 %) and RMSE from 79.403 to 11.299 kWh (−85.8 %), and it beat the baseline in all ten regions. Its typical error is ≈ 1.3 % of the mean regional hourly consumption (≈ 574 kWh), against ≈ 7.4 % for the baseline. At each region's peak hour it stayed within 0.06–62 kWh of actual but underestimated eight of the ten peaks (by up to ≈ 6 %), so keep a ≈ 6 % margin on peak hours. Retrain regularly and re-run forecasts after late readings arrive.

Word count: 458.

6 Create and investigate continuous aggregates

6.1 Create the three aggregate views

Three initially unpopulated continuous aggregates were created over energy_readings, all with `timescaledb.materialized_only = TRUE` so that Section 7 measures stored data only:

View	Grouping	Stored columns	Expected rows
cagg_15m_region	15-minute bucket, region (inner join with meters)	bucket, region, energy_kwh_sum, power_kw_sum, reading_count	28 × 96 × 10 = 26,880
cagg_hour_region	CAT hour, region (inner join with meters)	same	28 × 24 × 10 = 6,720
cagg_day_meter	CAT day, meter_id	bucket, meter_id, energy_kwh_sum, power_kw_sum, reading_count	28 × 4,000 = 112,000

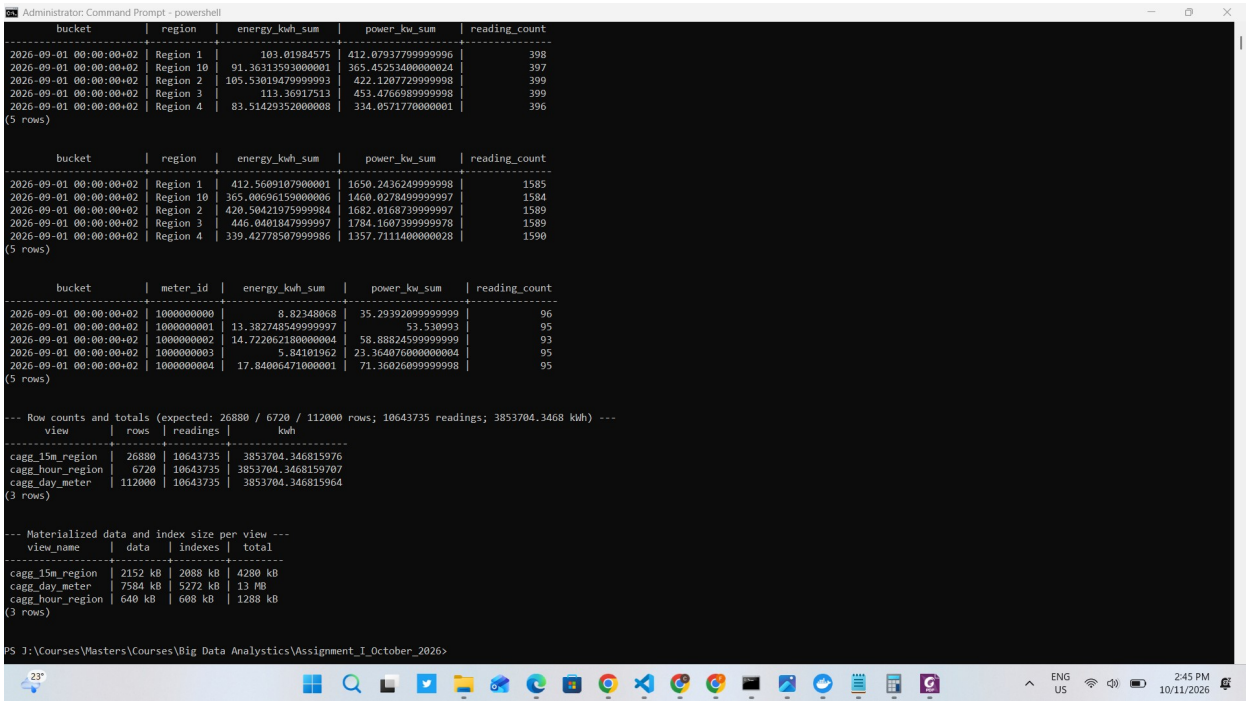


Figure 24 – Sample populated rows, row counts / totals and materialised size per view (11 Oct 2026, 2:45 PM).

View	Rows (expected)	Readings	Energy (kWh)	Data	Indexes	Total
cagg_15m_region	26,880 (26,880)	10,643,735	3,853,704.3468	2,152 kB	2,088 kB	4,280 kB
cagg_hour_region	6,720 (6,720)	10,643,735	3,853,704.3468	640 kB	608 kB	1,288 kB
cagg_day_meter	112,000 (112,000)	10,643,735	3,853,704.3468	7,584 kB	5,272 kB	13 MB

Verification. Each view holds the expected number of rows and reproduces the raw totals exactly: 10,643,735 readings and 3,853,704.35 kWh (differences below 10^{-7} kWh, i.e. floating-point rounding). The three views together occupy ≈ 18 MB (4,280 kB + 13 MB + 1,288 kB), dominated by the daily per-meter view. In the sample, each 15-minute region bucket on 1 September 00:00 contains 396–399 of the 400 possible readings, which illustrates the missing data of §2.2.

Refresh policies and jobs

Policies were added after the manual refresh of the whole period: `policy_refresh_continuous_aggregate` every **15 minutes**, scheduled, with a **3-day** start offset and a **1-hour** end offset. Because the offsets are relative to the database clock, the policies do not populate the September data – this was done by the manual refresh of the entire period.

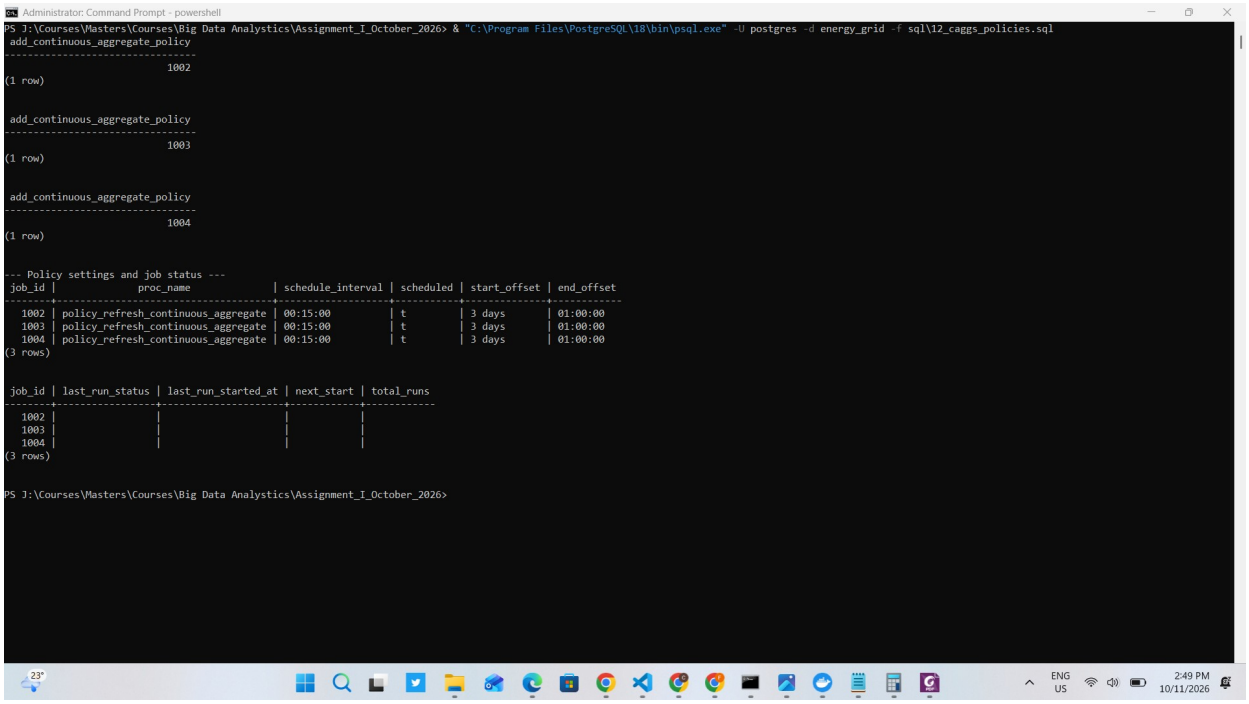


Figure 25 – Continuous-aggregate policies (jobs 1002–1004) and background job status (11 Oct 2026, 2:49 PM).

Audit and re-creation of the policies (12 October). When the background jobs were listed for the dashboard measurements (§7.4), the list of jobs no longer contained any `policy_refresh_continuous_aggregate` job: only the two compression policies (jobs 1000 and 1001) and the system jobs were registered (Figure 26). The three views were still present with `materialized_only = t` (Figure 27). The three refresh policies were therefore re-created with the same settings (3-day start offset, 1-hour end offset, 15-minute schedule) as jobs 1008, 1009 and 1010 (Figure 28); the final list shows seven scheduled jobs (Figure 29). The stored September data were not affected, because the policies only cover the last three days before the database clock.

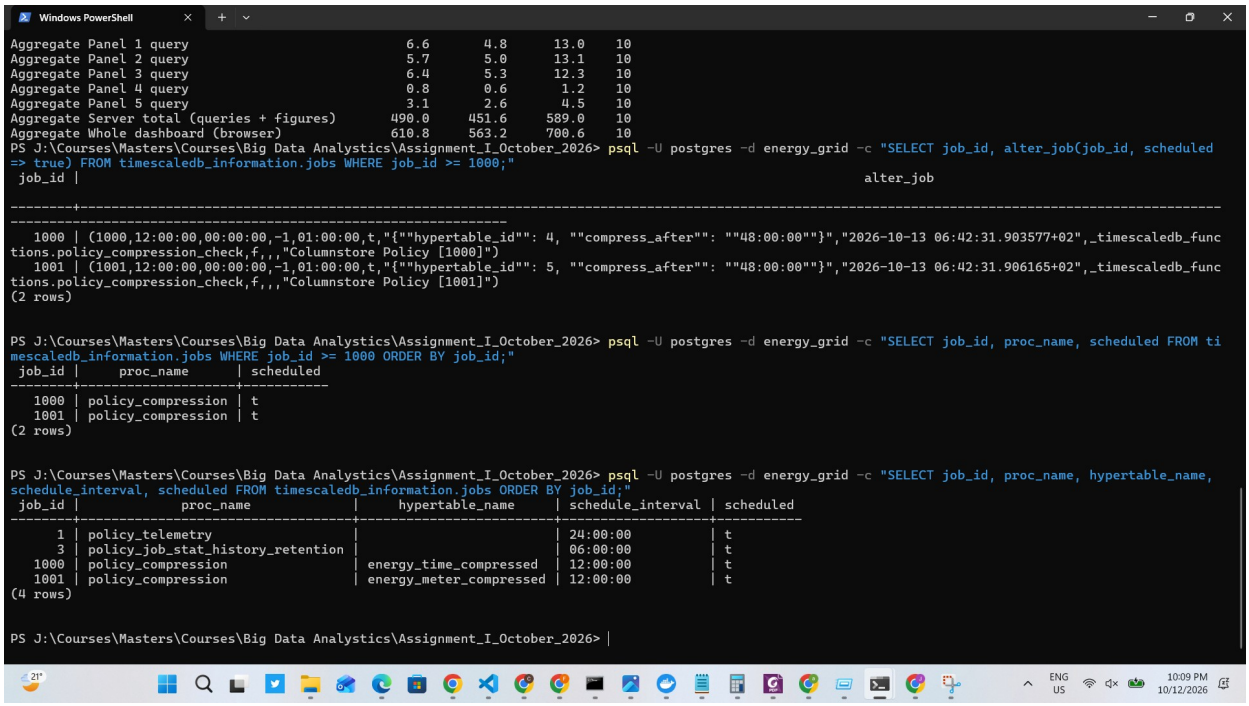


Figure 26 – Audit of the background jobs before the correction: only jobs 1, 3, 1000 and 1001 are registered (12 Oct 2026, 10:09 PM).

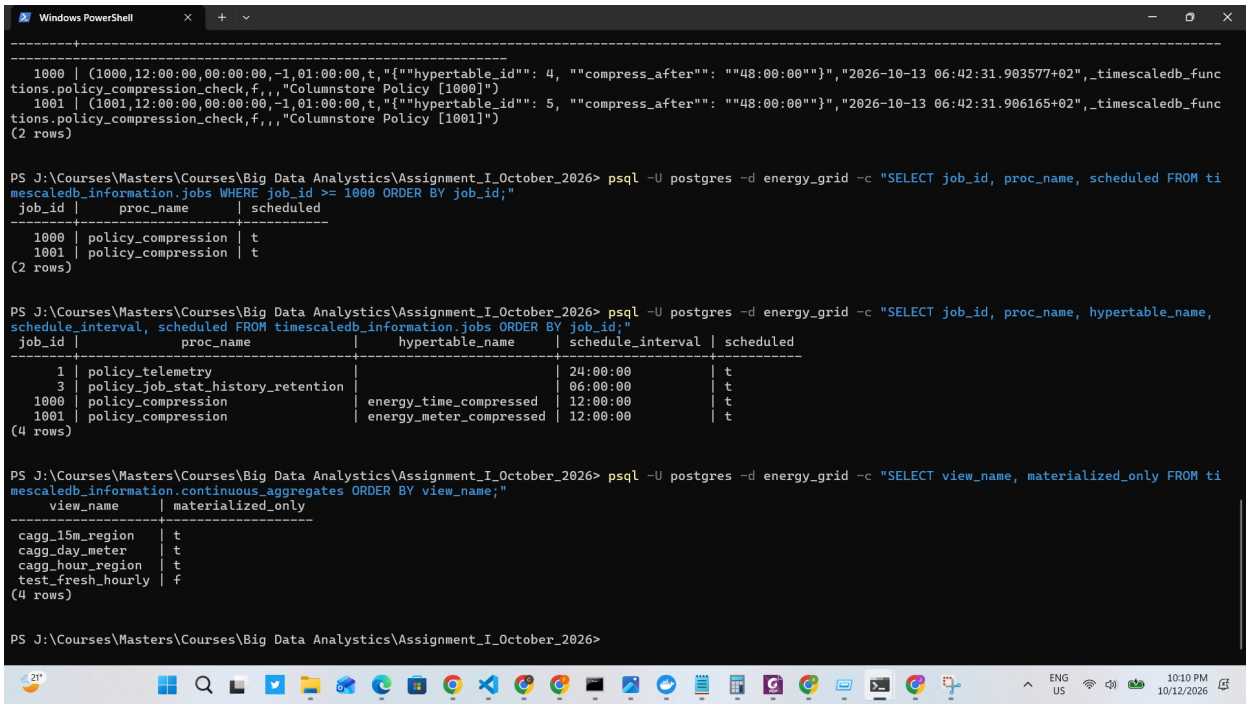


Figure 27 – Continuous aggregates: the three views are materialized-only (t); test_fresh_hourly is the §6.2 test view (12 Oct 2026, 10:10 PM).

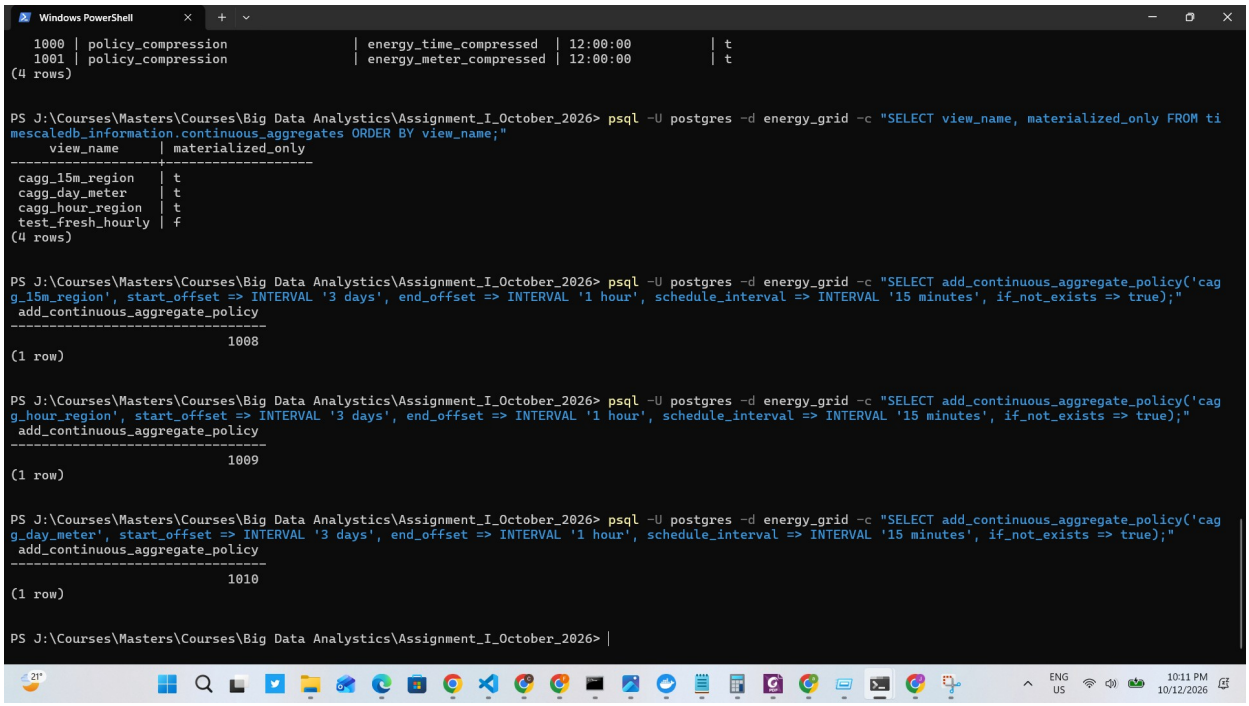


Figure 28 – Re-creation of the three refresh policies (jobs 1008, 1009, 1010; 3 days / 1 hour / 15 minutes) (12 Oct 2026, 10:11 PM).

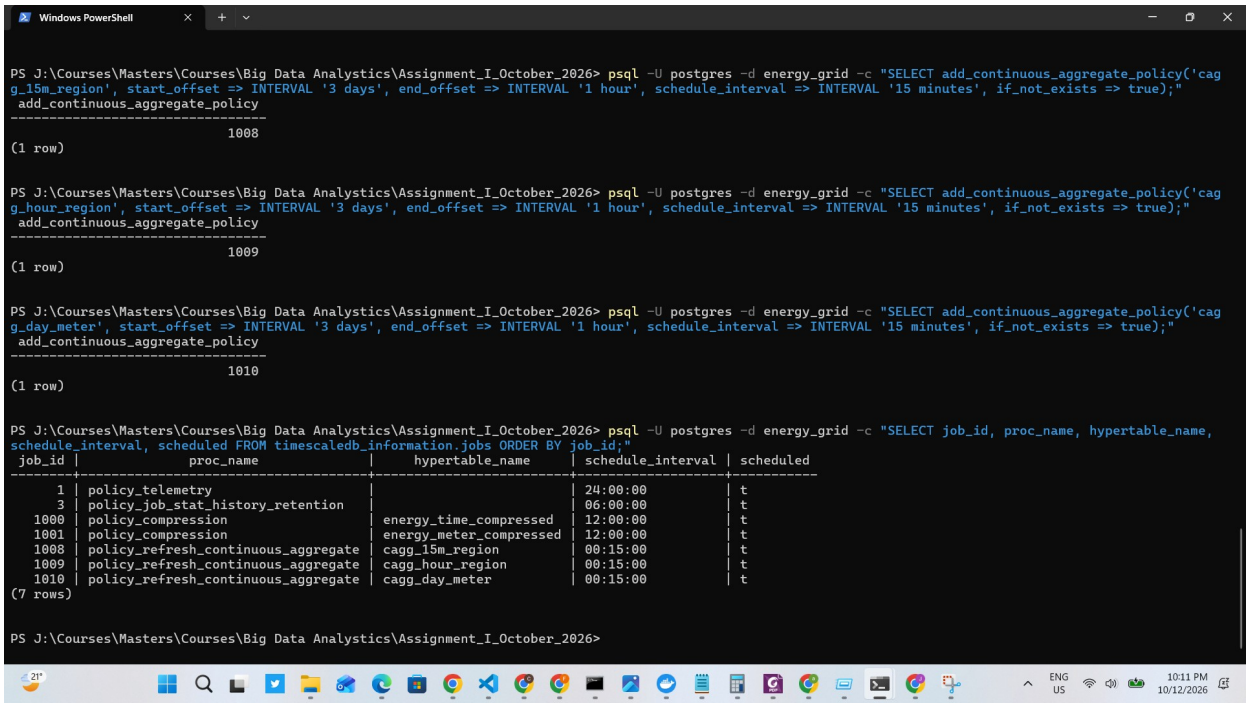


Figure 29 – All background jobs after the correction: three refresh policies and two compression policies, all scheduled (12 Oct 2026, 10:11 PM).

6.2 Test freshness and delayed arrivals

A separate test hypertable with an hourly continuous aggregate (`test_fresh_hourly`, created with `test_freshness.py`; automatic refresh off) is used to show how late inserts propagate. With H the start of the latest completed CAT hour, a 1 kWh reading is inserted in the hour H − 48 h and refreshed manually; a further 0.5 kWh reading is then inserted in that hour and a 2 kWh reading in hour H, and the raw table and the aggregate are compared before and after enabling real-time aggregation and before and after a manual refresh of both hours.

Explanation of the stages. With real-time aggregation off, the aggregate shows only what was materialised: the late 0.5 kWh reading stays hidden and hour H returns no bucket (an absent bucket is not a zero reading). Enabling real-time aggregation adds the not-yet-materialised rows of the recent region, which is why hour H becomes visible, but it does not revise already materialised buckets: the older hour still shows 1 kWh until a refresh covers it. After a manual refresh of both complete hours the aggregate matches the raw table: 1.5 kWh in two rows for the older hour and 2 kWh in one row for hour H. The §6.1 policy refreshes only the window from three days to one hour before the database clock, so hour H (inside the one-hour end offset) cannot be refreshed by the policy, whereas the older hour (48 hours old) can.

7 Build and compare dashboards

7.1 Build the Raw dashboard

The **Raw** dashboard (Dash/Plotly, `python dashboard_app.py Raw`, served on 127.0.0.1:8051) has five panels – hourly energy by region, completeness by region, daily network energy, network energy per 7-day period and daily energy of the selected meter – and filters for the CAT days (start and end day, both included) and the meter. All measurement queries read `energy_readings` directly and join `meters` where needed (panel queries in `dashboard_queries.py`). Every panel is limited to the selected date range; energy is labelled in kWh and completeness in percent.

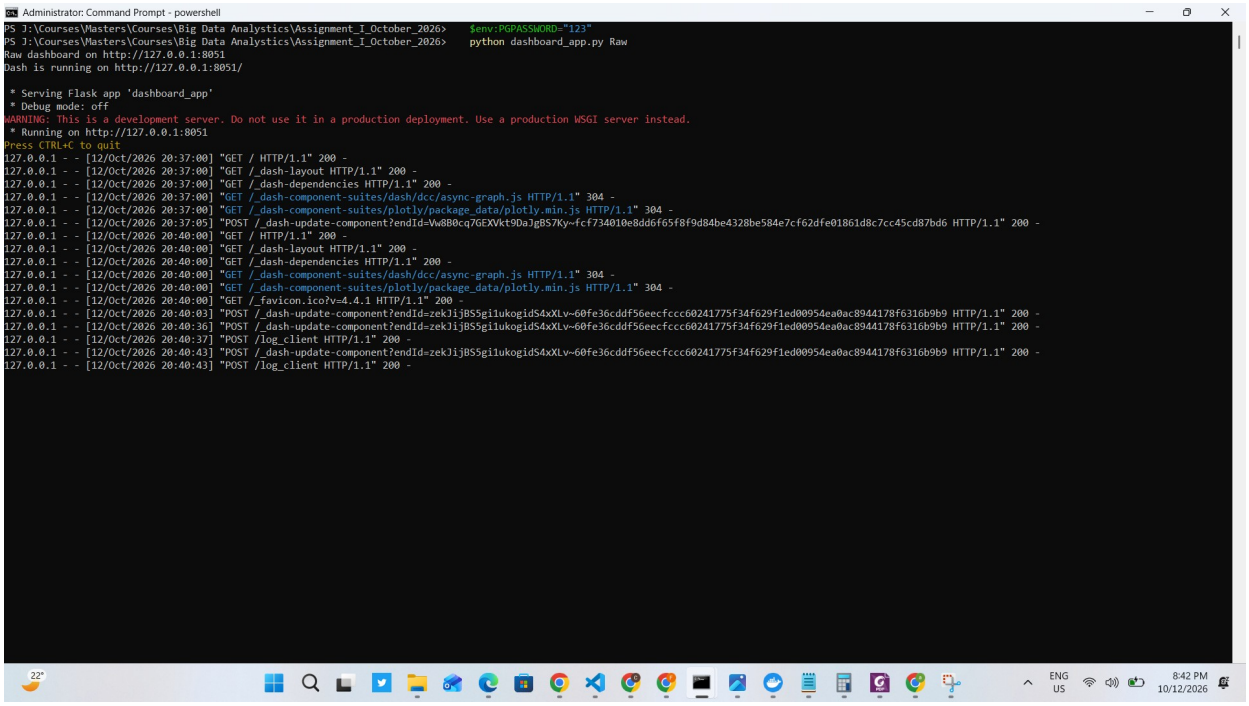


Figure 30 – Command that starts the Raw dashboard, with the server running on port 8051 (12 Oct 2026, 8:42 PM).

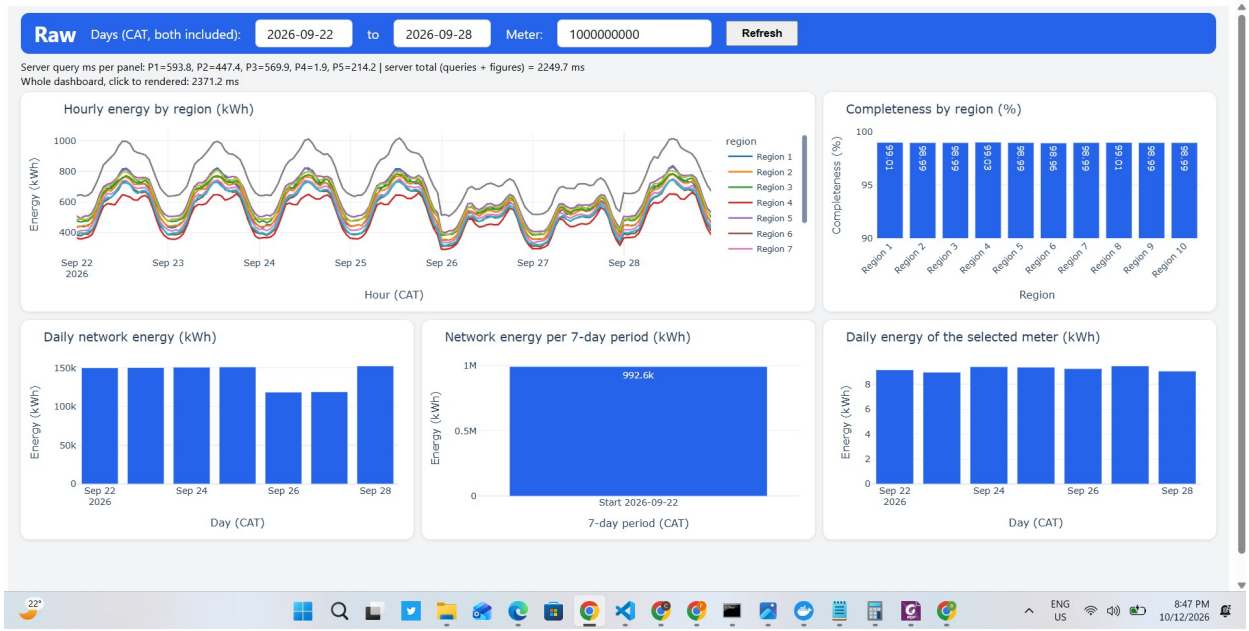


Figure 31 – Raw dashboard, 22–28 September 2026, meter 1000000000 (12 Oct 2026, 8:47 PM).

Measured on this refresh: server query times P1 = 593.8 ms, P2 = 447.4 ms, P3 = 569.9 ms, P4 = 1.9 ms, P5 = 214.2 ms; server total (queries + figures) = 2,249.7 ms; whole dashboard, click to rendered = 2,371.2 ms.

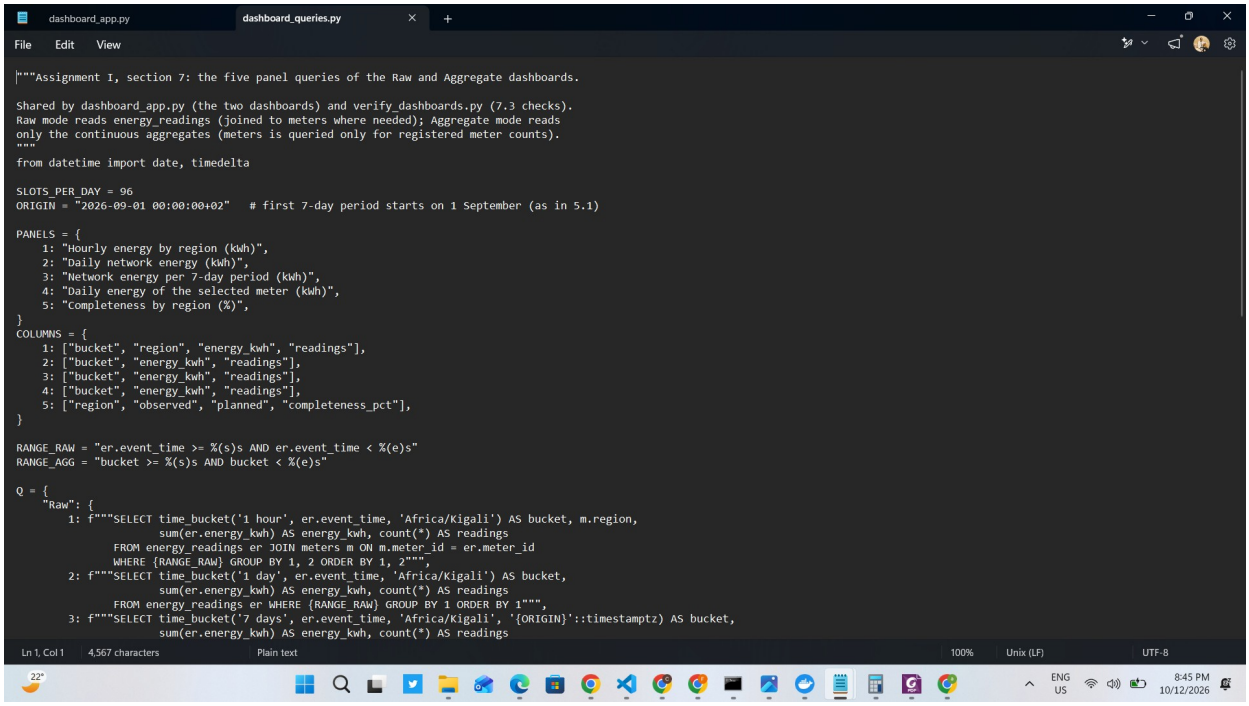


Figure 32 – Raw panel queries in dashboard_queries.py (12 Oct 2026, 8:45 PM).

7.2 Build the Aggregate dashboard

The **Aggregate** dashboard (`python dashboard_app.py Aggregate`, 127.0.0.1:8052) has the same panels, layout, filters and date range, but its queries read `cagg_hour_region` (hourly regional energy), `cagg_15m_region` (daily and weekly network energy and observed completeness counts) and `cagg_day_meter` (meter history). Energy sums and reading counts are summed across buckets; planned readings are derived from the registered meters (400 per region) and the selected period. Only the meters table is read for labels and counts.

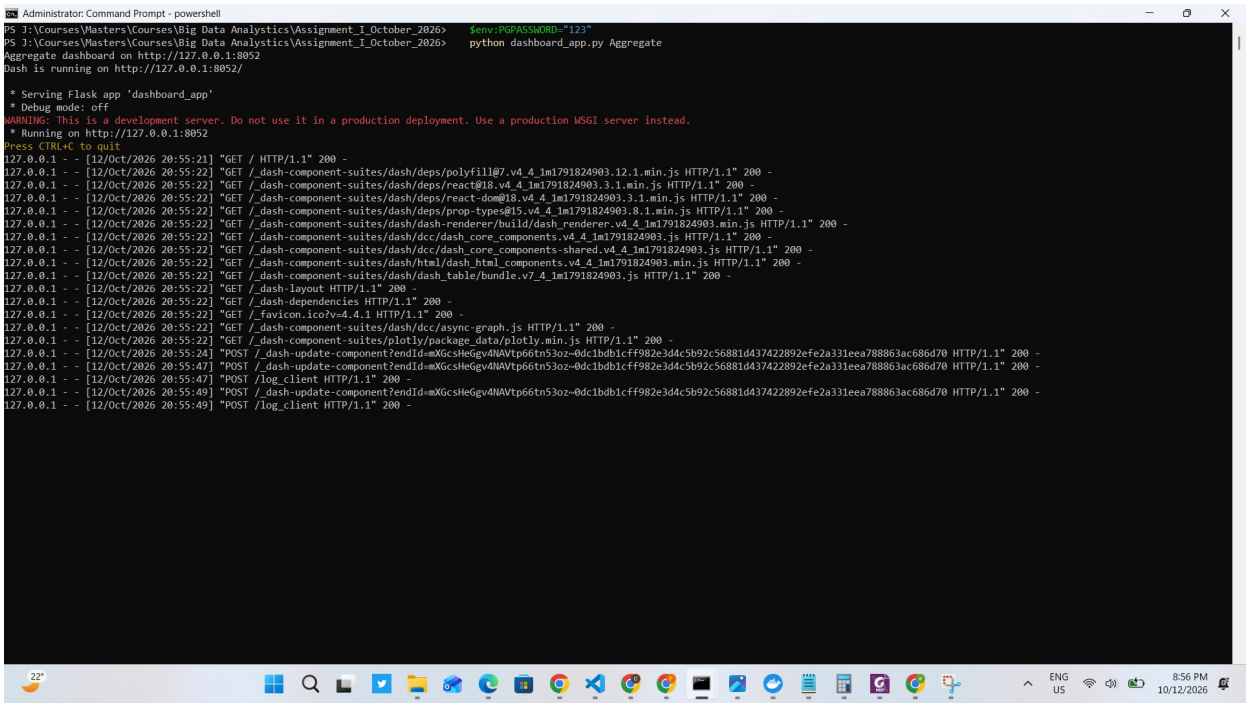


Figure 33 – Command that starts the Aggregate dashboard, with the server running on port 8052 (12 Oct 2026, 8:56 PM).

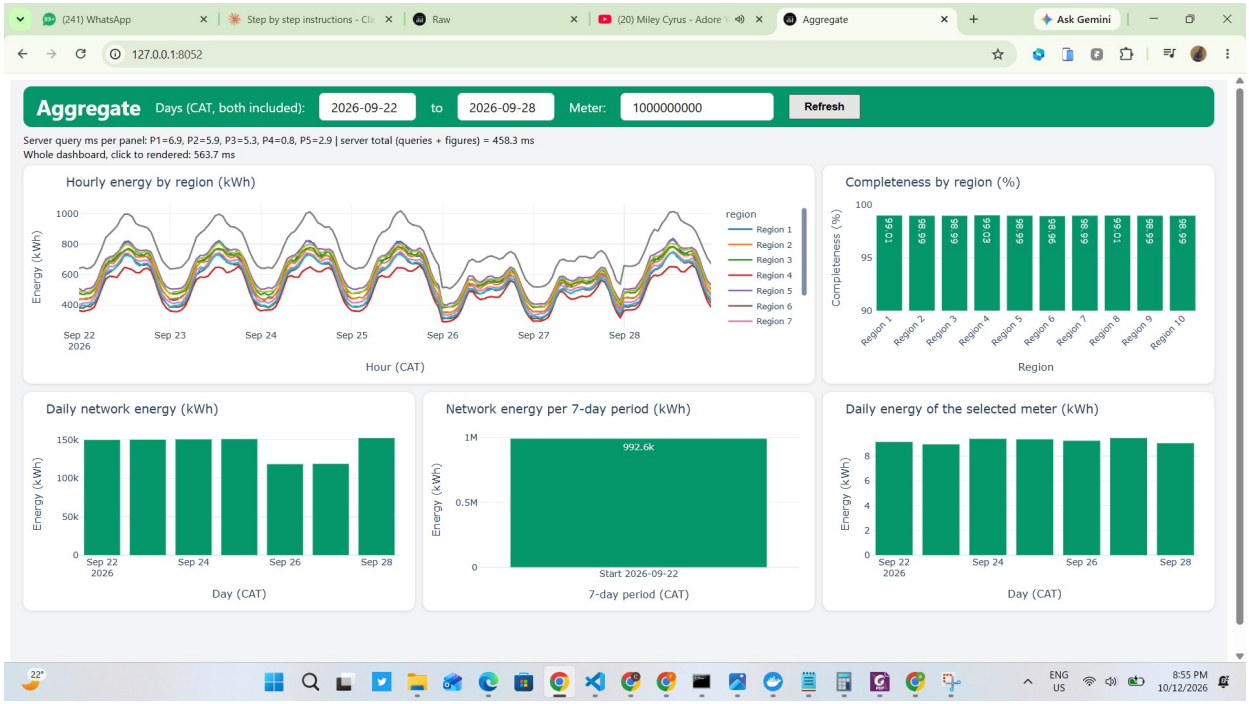
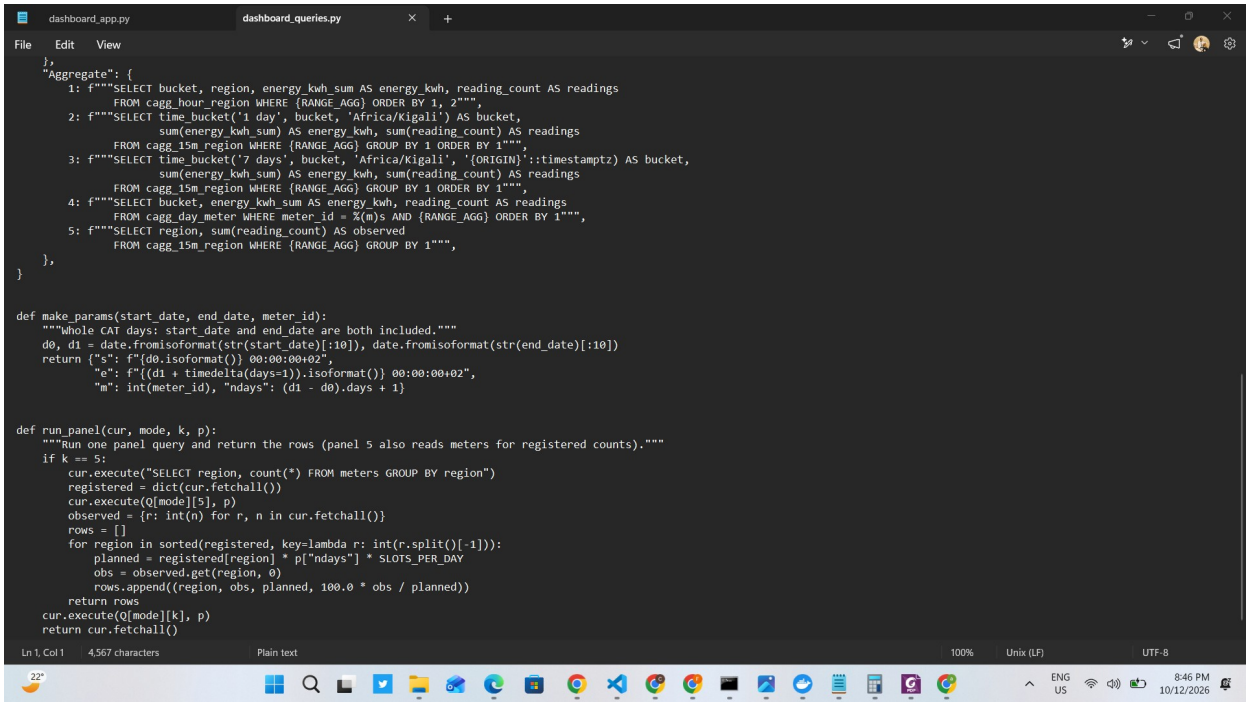


Figure 34 – Aggregate dashboard, same filters (12 Oct 2026, 8:55 PM).

Measured on this refresh: P1 = 6.9 ms, P2 = 5.9 ms, P3 = 5.3 ms, P4 = 0.8 ms, P5 = 2.9 ms; server total = 458.3 ms; whole dashboard = 563.7 ms.



```
    },
    "Aggregate": {
        1: f"""SELECT bucket, region, energy_kwh_sum AS energy_kwh, reading_count AS readings
               FROM cagg_hour_region WHERE {RANGE_AGG} ORDER BY 1, 2""",
        2: f"""SELECT time_bucket('1 day', bucket, 'Africa/Kigali') AS bucket,
                      sum(energy_kwh_sum) AS energy_kwh, sum(reading_count) AS readings
               FROM cagg_15m_region WHERE {RANGE_AGG} GROUP BY 1 ORDER BY 1""",
        3: f"""SELECT time_bucket('7 days', bucket, 'Africa/Kigali', '{ORIGIN}'::timestamptz) AS bucket,
                      sum(energy_kwh_sum) AS energy_kwh, sum(reading_count) AS readings
               FROM cagg_15m_region WHERE {RANGE_AGG} GROUP BY 1 ORDER BY 1""",
        4: f"""SELECT bucket, energy_kwh_sum AS energy_kwh, reading_count AS readings
               FROM cagg_day_meter WHERE meter_id = %(m)s AND {RANGE_AGG} ORDER BY 1""",
        5: f"""SELECT region, sum(reading_count) AS observed
               FROM cagg_15m_region WHERE {RANGE_AGG} GROUP BY 1""",
    },
}


def make_params(start_date, end_date, meter_id):
    """Whole CAT days: start_date and end_date are both included."""
    d0, d1 = date.fromisoformat(str(start_date)[:10]), date.fromisoformat(str(end_date)[:10])
    return {"s": f"{d0.isoformat()} 00:00:00+02",
            "e": f"{(d1 + timedelta(days=1)).isoformat()} 00:00:00+02",
            "m": int(meter_id), "ndays": (d1 - d0).days + 1}


def run_panel(cur, mode, k, p):
    """Run one panel query and return the rows (panel 5 also reads meters for registered counts)."""
    if k == 5:
        cur.execute("SELECT region, count(*) FROM meters GROUP BY region")
        registered = dict(cur.fetchall())
        cur.execute(Q[mode][5], p)
        observed = {r: int(n) for r, n in cur.fetchall()}
        rows = []
        for region in sorted(registered, key=lambda r: int(r.split()[-1])):
            planned = registered[region] * p["ndays"] * SLOTS_PER_DAY
            obs = observed.get(region, 0)
            rows.append((region, obs, planned, 100.0 * obs / planned))
        return rows
    cur.execute(Q[mode][k], p)
    return cur.fetchall()
```

Figure 35 – Aggregate panel queries and parameter handling in dashboard_queries.py (12 Oct 2026, 8:46 PM).

7.3 Verify matching results

Both dashboards were set to 22 September 00:00 – 29 September 00:00 (exclusive), all regions, meter 1000000000, and displayed side by side (Raw in blue, Aggregate in green).

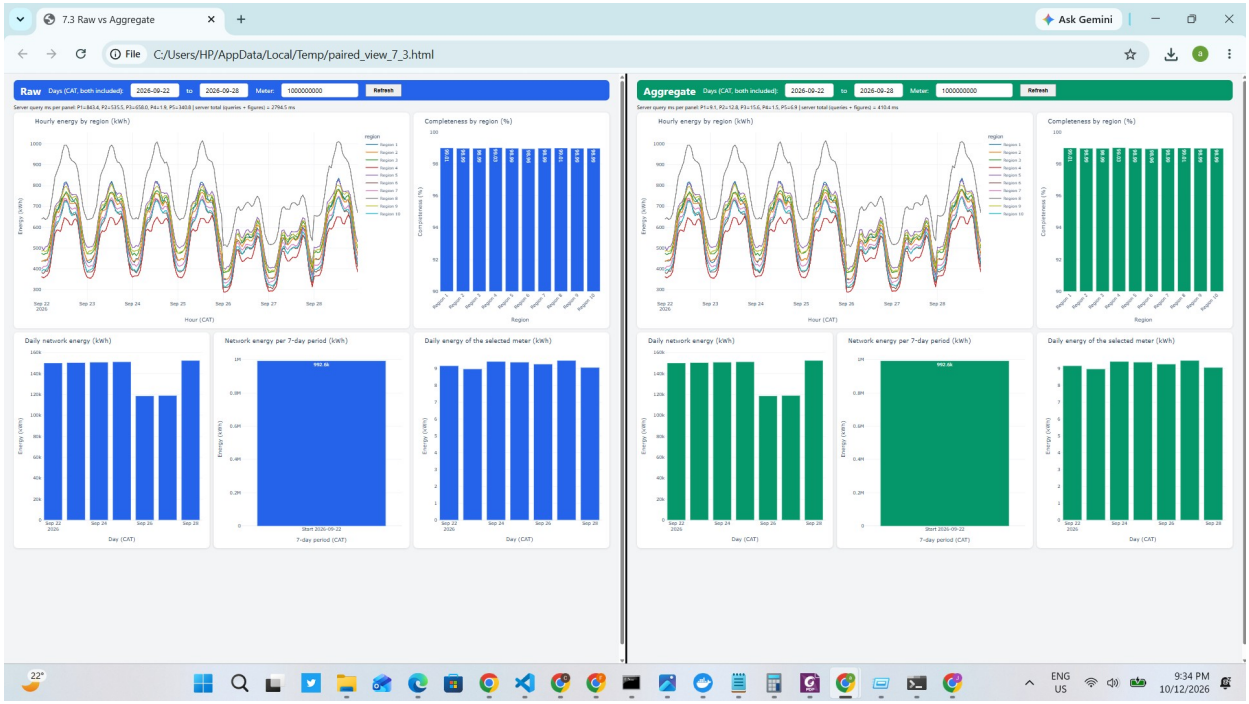


Figure 36 – Paired dashboard outputs: Raw (left) and Aggregate (right) (12 Oct 2026, 9:34 PM).

The data behind every panel were queried from both modes and compared by bucket, region or meter with `verify_dashboards.py`. The criteria are identical keys and reading counts, energy differences no greater than 0.001 kWh and completeness differences no greater than 0.000001 percentage points.

Panel	Rows compared	Keys identical	Counts identical	Max |ΔE| (kWh)	Max |ΔC| (pp)	Result
1 Hourly energy by region	1,680	True	True	0.000000000	0.000000000	PASS
2 Daily network energy	7	True	True	0.000000005	0.000000000	PASS
3 Network energy per 7-day period	1	True	True	0.000000012	0.000000000	PASS
4 Daily energy of the selected meter	7	True	True	0.000000000	0.000000000	PASS
5 Completeness by region	10	True	True	0.000000000	0.000000000	PASS

All five panels pass. The residual energy differences (at most 1.2×10^{-8} kWh) are floating-point rounding caused by summing the same readings in a different order; the keys and reading counts are identical and the completeness values agree exactly, so no correction was needed before timing.

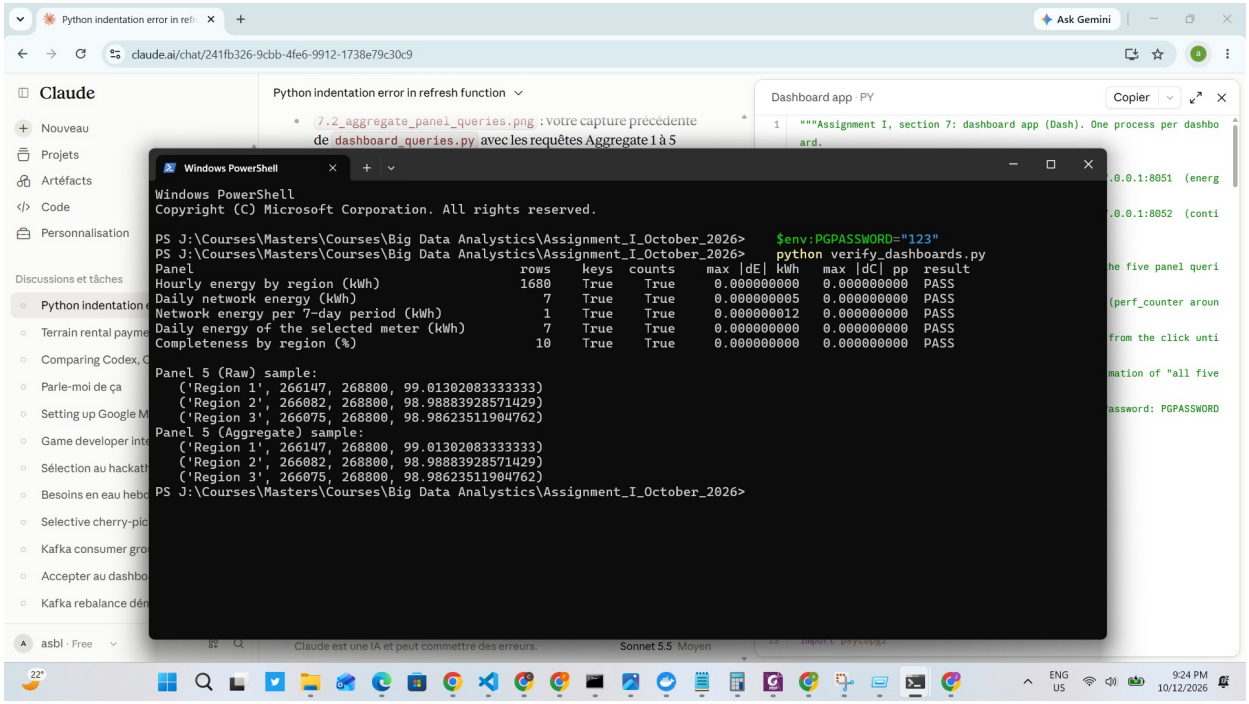


Figure 37 – Numerical check of the five panels, Raw versus Aggregate (verify_dashboards.py) (12 Oct 2026, 9:24 PM).

7.4 Measure performance and judge the trade-off

Method. Both dashboards ran with the same filters and dates as in §7.3 (22–28 September, meter 1000000000), on the same machine, browser and database connection, one server at a time, with no query-result caching. Each mode was measured with one warm-up refresh followed by ten measured refreshes. The per-panel query duration is taken on the server with `time.perf_counter()` around execute and fetchall; the whole-dashboard time is taken in the browser with `performance.now()` from the click until the response has arrived and two animation frames have passed. Every refresh is appended to `dashboard_timings.csv`; `summarize_dashboard_timings.py` computes the statistics. Before the series the compression jobs 1000 and 1001 were paused; the query on jobs with job_id ≥ 1000 returned no refresh job (Figure 38), so no refresh or compression job ran during the measurements. The Raw warm-up refresh took 17,200.9 ms in the browser (cold load) and is excluded.

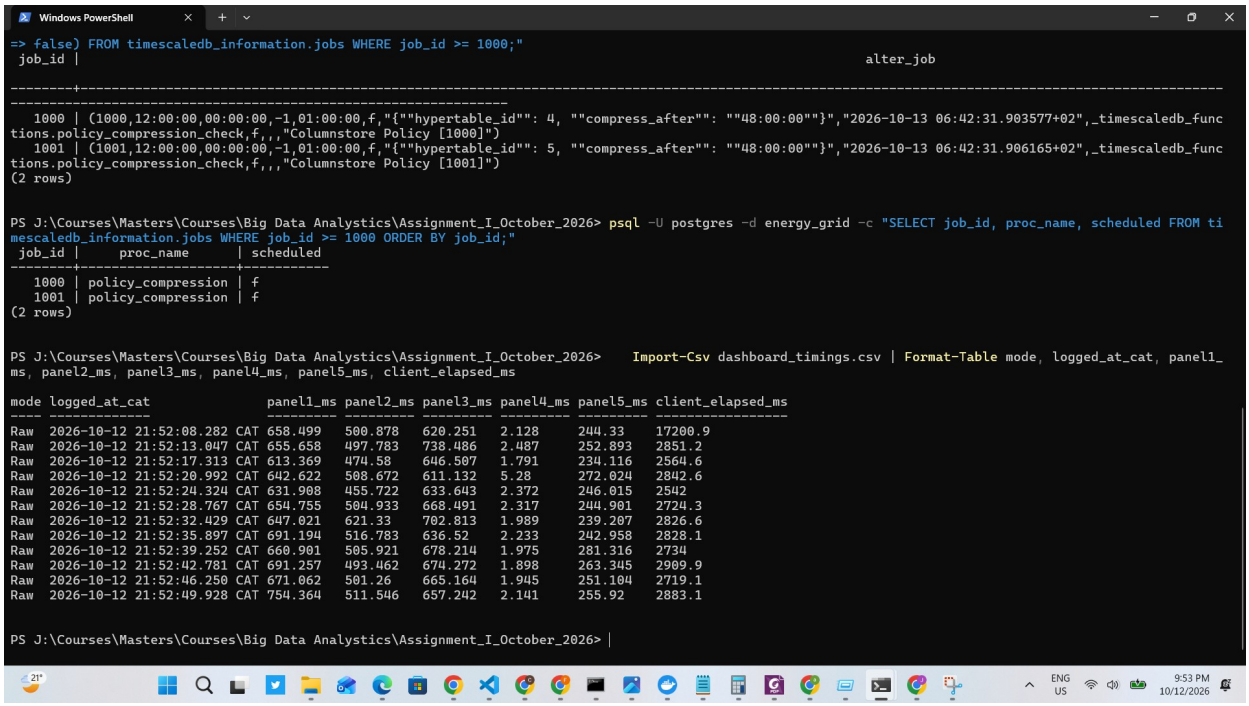


Figure 38 – Jobs paused (scheduled = f) and the eleven Raw refreshes recorded in dashboard_timings.csv: one warm-up and ten measured (12 Oct 2026, 9:53 PM).

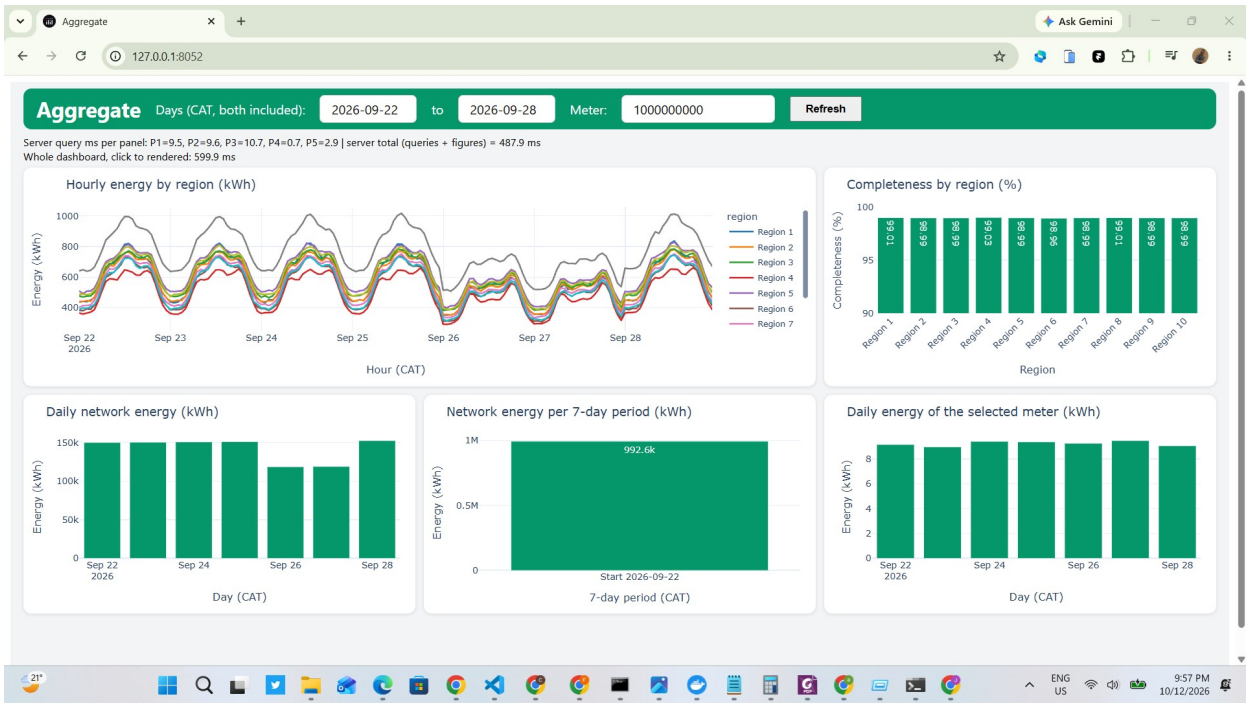


Figure 39 – Aggregate dashboard during the measured series (12 Oct 2026, 9:57 PM).

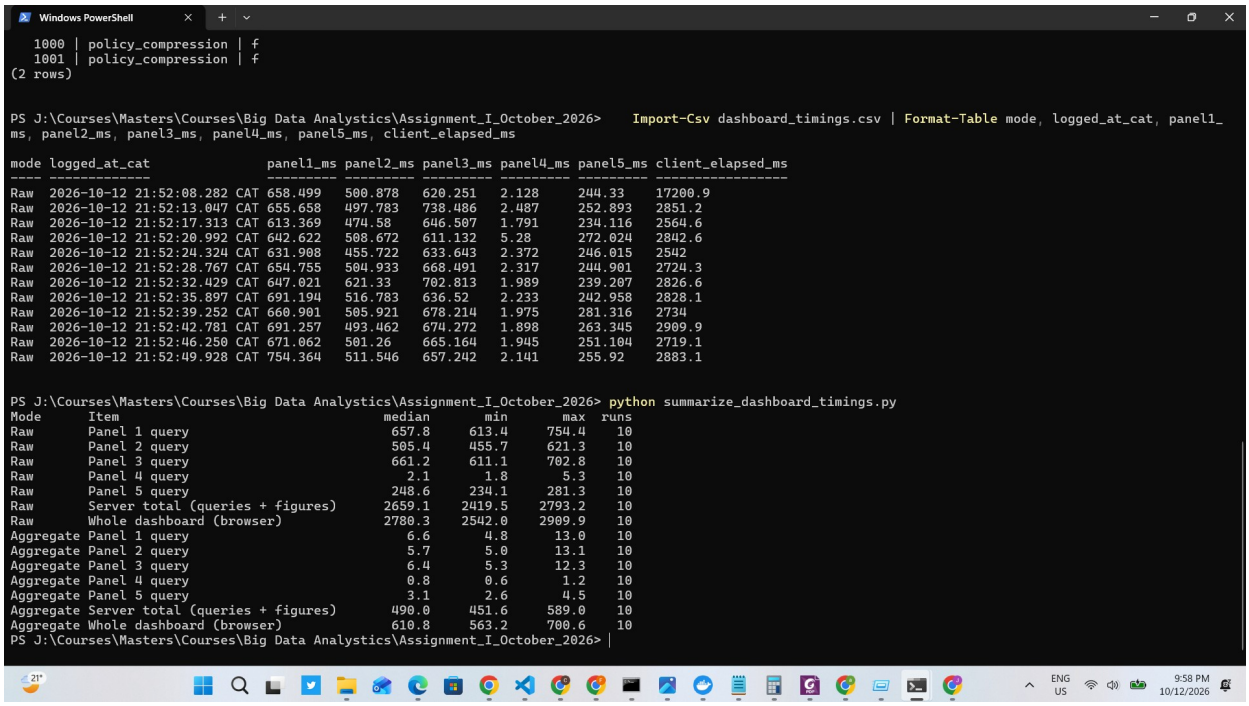


Figure 40 – Timing summary of the ten measured refreshes per mode (summarize_dashboard_timings.py) (12 Oct 2026, 9:58 PM).

Milliseconds, 10 measured refreshes per mode:

Panel / metric	Raw median	Raw min–max	Aggregate median	Aggregate min–max	Speed-up (median)
P1 Hourly energy by region	657.8	613.4 – 754.4	6.6	4.8 – 13.0	≈ 99.7×
P2 Daily network energy	505.4	455.7 – 621.3	5.7	5.0 – 13.1	≈ 88.7×
P3 Energy per 7-day period	661.2	611.1 – 702.8	6.4	5.3 – 12.3	≈ 103.3×
P4 Daily energy, selected meter	2.1	1.8 – 5.3	0.8	0.6 – 1.2	≈ 2.6×
P5 Completeness by region	248.6	234.1 – 281.3	3.1	2.6 – 4.5	≈ 80.2×
Server total (queries + figures)	2,659.1	2,419.5 – 2,793.2	490.0	451.6 – 589.0	≈ 5.4×
Whole dashboard (browser)	2,780.3	2,542.0 – 2,909.9	610.8	563.2 – 700.6	≈ 4.6×

Storage and refresh cost (from §6.1). The three aggregates need ≈ 18 MB (4,280 kB + 13 MB + 1,288 kB) on top of the raw hypertable, one initial refresh of the whole September period, and one 15-minute refresh job per view (jobs 1008–1010).

Judgement. The speed change justifies the extra storage and refresh work. The aggregates are tiny compared with the raw hypertable, return numerically identical results (§7.3) and cut the median whole-dashboard latency from 2,780.3 ms to 610.8 ms (−78 %, ≈ 4.6×) and the server query time of the four all-meter panels by a factor of 80 to 103. The end-to-end gain is smaller than the query gain because a fixed non-SQL part is common to both modes: the server total minus the panel medians is ≈ 580 ms for Raw and ≈ 470 ms for Aggregate (building the figures), and the browser adds ≈ 121 ms in both. The single-meter panel P4 was already cheap in Raw mode (index on meter_id, event_time) and gains only ≈ 2.6×. The price is staleness (materialised-only views are only as fresh as the last refresh), the need to refresh late-arriving data, and fixed grains of 15 minutes, one hour and one day – ad-hoc questions at other grains still need the raw table.

Jobs after the measurements. The compression jobs were resumed after the series (Figure 41) and the three refresh policies were re-created and enabled (§6.1, Figures 28–29).

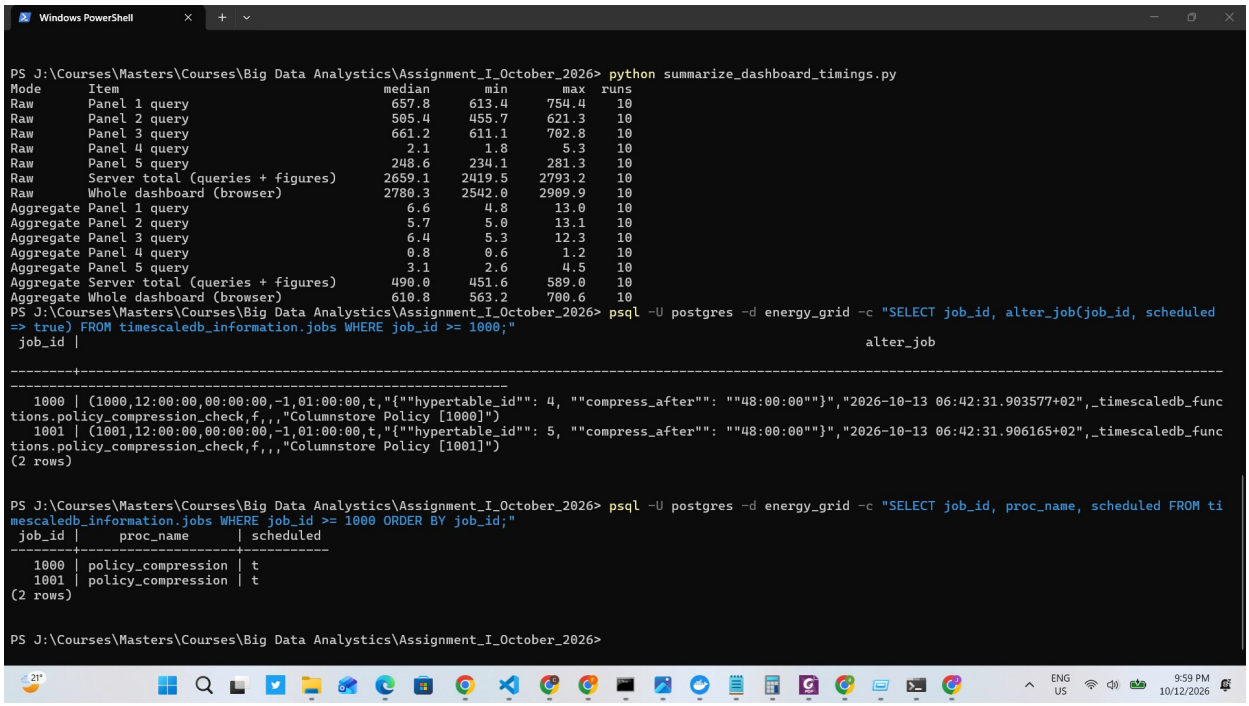


Figure 41 – Compression jobs resumed (scheduled = t) after the measurements, with the timing summary above (12 Oct 2026, 9:59 PM).

8 Forecast the next day

Goal: forecast the hourly energy (kWh) of each of the 10 regions for the next day, from a forecast origin at 00:00 CAT of the target day. This is an offline forecast on observed consumption; the remaining missing readings are a limitation. All steps are in `section8_forecast.py`.

8.1 Prepare chronological data and features

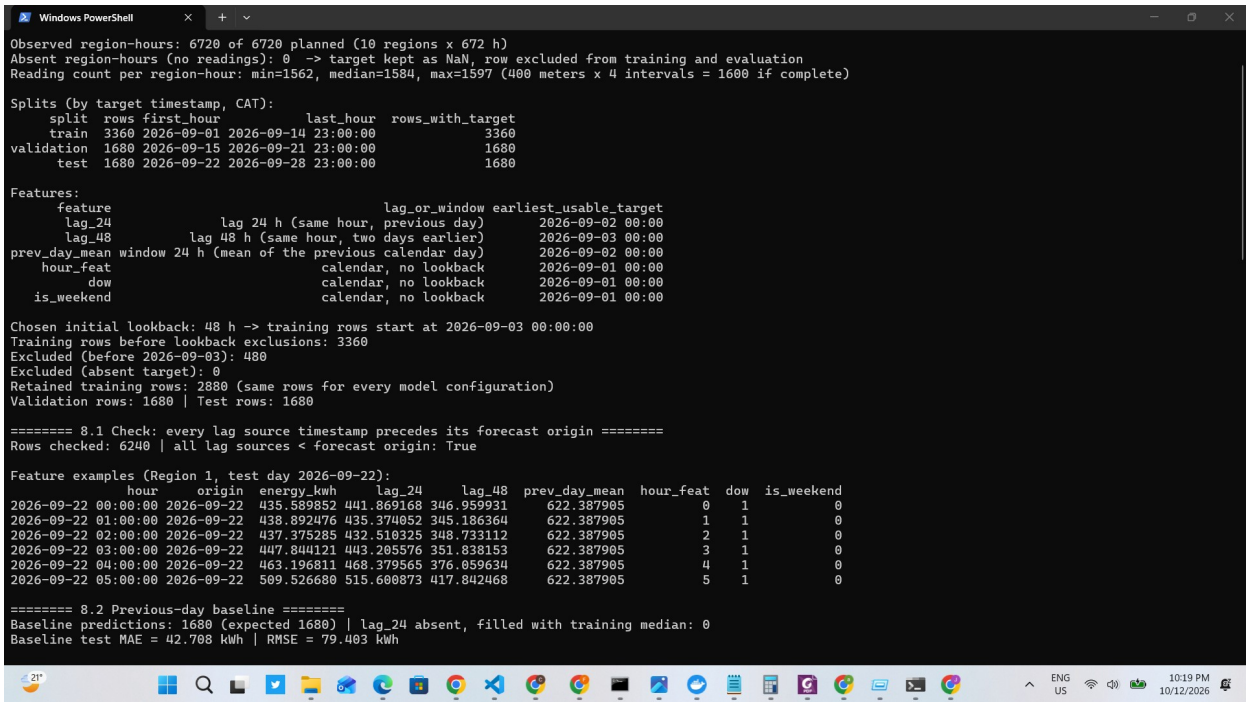


Figure 42 – Dataset preparation, splits, features, lookback exclusions, lag checks and baseline run (12 Oct 2026, 10:19 PM).

Item	Result
Region-hours observed	6,720 of 6,720 planned (10 regions × 672 h); absent region-hours: 0 (a NaN target would be excluded)
Readings per region-hour	min 1,562 · median 1,584 · max 1,597 (1,600 if complete: 400 meters × 4 intervals)
Train	3,360 rows · 2026-09-01 00:00 → 2026-09-14 23:00
Validation	1,680 rows · 2026-09-15 00:00 → 2026-09-21 23:00
Test	1,680 rows · 2026-09-22 00:00 → 2026-09-28 23:00
Training rows before lookback exclusions	3,360
Excluded (before 2026-09-03 00:00; initial lookback 48 h)	480 (absent target: 0)
Retained training rows (identical for every model configuration)	2,880

Feature	Lag / window	Earliest usable target
lag_24	24 h (same hour, previous day)	2026-09-02 00:00
lag_48	48 h (same hour, two days earlier)	2026-09-03 00:00
prev_day_mean	24 h window (mean of the previous calendar day)	2026-09-02 00:00
hour_feat	calendar, no lookback	2026-09-01 00:00
dow	calendar, no lookback	2026-09-01 00:00
is_weekend	calendar, no lookback	2026-09-01 00:00

Leakage check. For all 6,240 checked rows (2,880 training + 1,680 validation + 1,680 test) every lag source timestamp precedes its forecast origin (True). A preceding hour inside the forecast day is never used, because it would not be available at 00:00. Because lag_48 requires two days of history, the first 480 training rows (1–2 September) are dropped for all configurations so that models are compared on identical data; preprocessing is fitted on the training split only.

Feature example (Region 1, test day 22 September 2026): for the 00:00 target, energy_kwh = 435.59 kWh, lag_24 = 441.87 kWh, lag_48 = 346.96 kWh and prev_day_mean = 622.39 kWh; dow = 1 (Tuesday) and is_weekend = 0. The same day shows the same pattern at 05:00 (509.53 kWh vs. lag_24 515.60 kWh).

8.2 Evaluate the previous-day baseline

At 00:00 CAT of each test day, each of the 24 hourly values of each of the ten regions is predicted by the consumption of the same hour on the previous day. This gives 1,680 test predictions (no lag_24 value is absent, so no fill with the training median was required); sample predictions are shown at the top of Figure 43 (§8.3).

Method	Predictions	MAE (kWh)	RMSE (kWh)
Previous-day baseline (test, 22–28 Sept)	1,680	42.708	79.403

The mean regional hourly consumption is ≈ 574 kWh over the month (3,853,704 kWh / 6,720 region-hours), so the baseline error is ≈ 7.4 % of consumption. RMSE being almost twice the MAE shows that a few large errors dominate – typically the transitions between a weekend and a weekday, where the previous day has a different level (e.g. Monday forecasts based on Sunday).

8.3 Train and evaluate one regression model

Model. One regression model, scikit-learn *HistGradientBoostingRegressor* (random_state = 0), was trained in three configurations on the same 2,880 retained training rows. Shared predictors: region (one-hot encoded), lag_24, lag_48, prev_day_mean, hour_feat, dow and is_weekend. Preprocessing (one-hot encoding and median imputation of the numeric predictors) is fitted on the training split only.

Config ID	Model settings	Train rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
C1	max_iter = 100, learning_rate = 0.05, max_depth = 3	2,880	1,680	15.622	21.857
C2	max_iter = 300, learning_rate = 0.05, max_depth = 5	2,880	1,680	7.149	10.516
C3 (selected)	**max_iter = 300, learning_rate = 0.10, max_depth = None**	2,880	1,680	**6.246**	**8.654**

Configuration C3 has the lowest validation MAE and was frozen (fitted on the training split only) before testing. It was then used to produce the same 1,680 test predictions as the baseline, using only information available before each daily forecast origin: earlier completed test days supply later lag values but are never used to refit the model or the preprocessing.

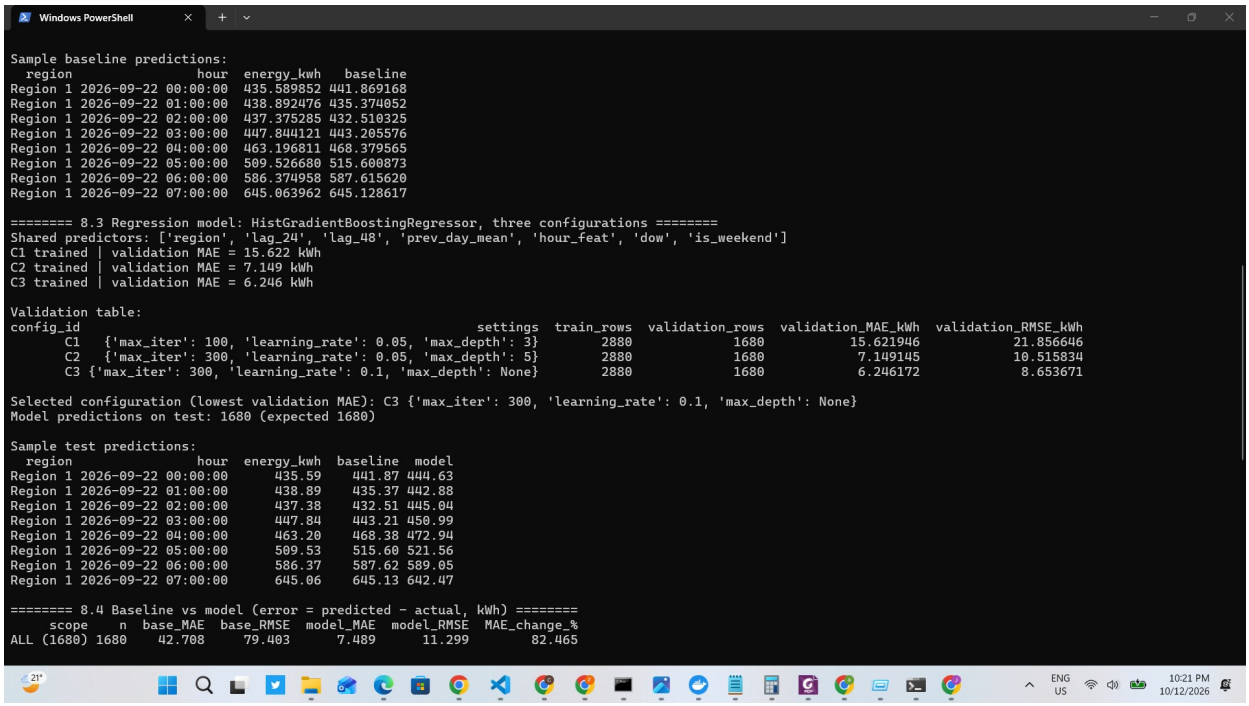


Figure 43 – Training of the three configurations, validation table, selected configuration, 1,680 test predictions and samples (12 Oct 2026, 10:21 PM).

8.4 Judge forecast usefulness

Each test day is forecast at 00:00 for 24 hourly values per region (1,680 predictions in total). Prediction error is predicted minus actual kWh; MAE is the mean of its absolute values and RMSE the square root of its mean square, both in kWh.

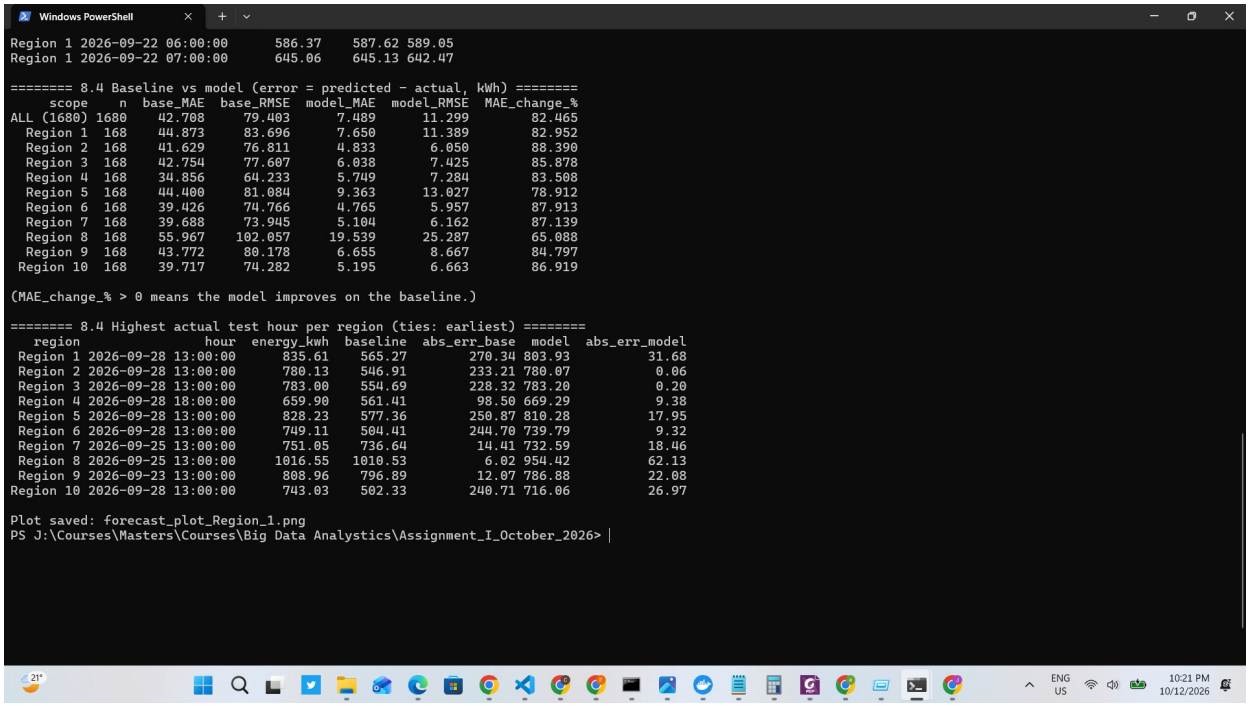


Figure 44 – Baseline versus model metrics (overall and per region) and highest actual test hour of each region (12 Oct 2026, 10:21 PM).

MAE and RMSE in kWh, test split (22–28 September):

Scope	n	Baseline MAE	Baseline RMSE	Model MAE	Model RMSE	MAE change
All regions	1,680	42.708	79.403	7.489	11.299	−82.5 %
Region 1	168	44.873	83.696	7.650	11.389	−83.0 %
Region 2	168	41.629	76.811	4.833	6.050	−88.4 %
Region 3	168	42.754	77.607	6.038	7.425	−85.9 %
Region 4	168	34.856	64.233	5.749	7.284	−83.5 %
Region 5	168	44.400	81.084	9.363	13.027	−78.9 %
Region 6	168	39.426	74.766	4.765	5.957	−87.9 %
Region 7	168	39.688	73.945	5.104	6.162	−87.1 %
Region 8	168	55.967	102.057	19.539	25.287	−65.1 %
Region 9	168	43.772	80.178	6.655	8.667	−84.8 %
Region 10	168	39.717	74.282	5.195	6.663	−86.9 %

Highest actual consumption hour of each region in the test week (ties: earliest):

Region	Peak hour (CAT)	Actual (kWh)	Baseline pred.	Baseline abs. error	Model pred.	Model abs. error
Region 1	2026-09-28 13:00	835.61	565.27	270.34	803.93	31.68
Region 2	2026-09-28 13:00	780.13	546.91	233.21	780.07	0.06
Region 3	2026-09-28 13:00	783.00	554.69	228.32	783.20	0.20
Region 4	2026-09-28 18:00	659.90	561.41	98.50	669.29	9.38
Region 5	2026-09-28 13:00	828.23	577.36	250.87	810.28	17.95
Region 6	2026-09-28 13:00	749.11	504.41	244.70	739.79	9.32
Region 7	2026-09-25 13:00	751.05	736.64	14.41	732.59	18.46
Region 8	2026-09-25 13:00	1,016.55	1,010.53	6.02	954.42	62.13
Region 9	2026-09-23 13:00	808.96	796.89	12.07	786.88	22.08
Region 10	2026-09-28 13:00	743.03	502.33	240.71	716.06	26.97

Judgement

Does the model improve on the baseline? Yes. Observation: over the 1,680 test predictions MAE falls from 42.708 to 7.489 kWh (−82.5 %) and RMSE from 79.403 to 11.299 kWh (−85.8 %); the model has the lower MAE in every region, with improvements between −65.1 % (Region 8) and −88.4 % (Region 2). The test MAE (7.489 kWh) is ≈ 20 % higher than the validation MAE of the selected configuration (6.246 kWh), a mild degradation. Interpretation: the baseline ignores the calendar and so copies the wrong level when the previous day is a different type of day, whereas the model learns the weekday/weekend effect from dow, is_weekend and the lags.

Peak errors. Observation: on Monday 28 September the baseline copied the quiet Sunday and missed the regional peak by 228–270 kWh in six regions (Regions 1, 2, 3, 5, 6 and 10); the model's errors at the peak hours range from 0.06 to 62.13 kWh (at most ≈ 6.1 % of the peak, in Region 8, which also has the highest MAE, 19.539 kWh). The model underestimated eight of the ten peaks. The baseline was closer at the peaks of Regions 7, 8 and 9 (14.41, 6.02 and 12.07 kWh against 18.46, 62.13 and 22.08 kWh), which fall on weekdays that resemble the previous day. Interpretation: the model is far safer on days that follow a change of day type, but peak hours still need a safety margin of about 6 %.

Are the errors useful for planning? The model error is ≈ 1.3 % of the mean regional hourly consumption (≈ 574 kWh), against ≈ 7.4 % for the baseline, which is accurate enough for day-ahead scheduling of generation and purchases. Capacity decisions at the peak hour should use a margin because the largest errors are underestimates.

Missing readings and delayed arrivals. The hourly totals are built from 1,562–1,597 of the 1,600 possible readings per region-hour (≈ 0.2–2.4 % missing); targets and lags are affected in the same way, so the model learns the usual observed level, but a sudden loss of reporting would appear as a forecast error and not as a drop in consumption. This evaluation uses finalised hourly totals, as stated in the brief. In a live forecast at 00:00, about 2 % of the readings arrive 30 to 180 minutes late, so the last hours of the previous day are incomplete: the inputs lag_24 and prev_day_mean built from them would be slightly too low and the forecast biased downward. Running the forecast after the 180-minute late window, or recomputing it once late readings have arrived, removes this effect.

9 Submit the report and code

9.1 Organise the report

This report follows the order of the subsections of the brief. Each screenshot is embedded beside its subsection with the taskbar and clock visible.

9.2 Make the work reproducible

GitHub repository: https://github.com/Jocos12/Smart_Energy_Grid_Analystics.git

The repository and the ZIP contain the unchanged generator, the capture and ingestion programs, the SQL files, the analysis and model code, the recorded benchmark timings, the dashboard queries and configuration and the recorded environment details. Passwords are read from the PGPASSWORD environment variable; the code uses a placeholder default. The full generated dataset stays local.

Component	File / location
Generator (unchanged)	smart_meter_simulator.py
Metadata, pilot publisher and subscriber	load_meters.py, publisher.py, subscriber.py
Full dataset capture and load	load_full.py
Compression experiment	load_compress.py, compression_cutoff, compression_chunk_log
Chunk and compression benchmarks	bench_chunks.py, bench_compression.py, benchmark_chunks_runs / _summary, benchmark_compression_runs / _summary
Execution plans	explain_Q2_*, explain_Q4_* (energy_readings, _3h, _week, energy_time_compressed, energy_meter_compressed)
Analysis (§5)	analysis_5_2.py, analysis_5_3.py, analysis_5_4.py
SQL files	sql\05_chunk_checks.sql, sql\10_compression_sizes.sql, sql\12_caggs_policies.sql
Freshness test (§6.2)	test_freshness.py
Dashboards (§7)	dashboard_app.py (Raw on port 8051, Aggregate on port 8052), dashboard_queries.py, verify_dashboards.py, summarize_dashboard_timings.py, capture_7_3.py, dashboard_timings.csv
Forecast (§8)	section8_forecast.py
Screenshots	screenshots\

9.3 Document contributions

Member	Student ID
HITAYEZU James	101381
Joseph Courbon	101493
Mylene MANIRAKIZA	101403

All members are prepared to explain and present the group's work if called upon.